Evaluating Aspect-Based Sentiment Analysis in Healthcare Drug Reviews Across Machine Learning, Deep Neural Networks, and Transformer Models

Eun Soo Park

A Thesis Submitted in Partial Fulfillment of the

Requirements for the Degree of

Master of Science

In

Data Science

Minnesota State University, Mankato

Mankato, Minnesota

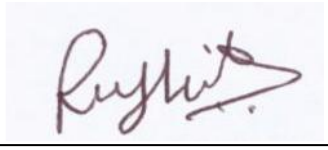
November 2025

11/10/2025

Title: Evaluating Aspect-Based Sentiment Analysis in Healthcare Drug Reviews Across Machine Learning, Deep Neural Networks, and Transformer Models

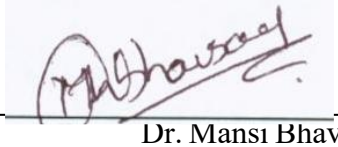
Student's Name: Eun Soo Park

This thesis has been examined and approved by the following members of the thesis committee.



Dr. Rushit Dave
Advisor

Dr. Rajeev Bukralia
Graduate Coordinator



Dr. Mansi Bhavsar,
Committee Member

Acknowledgments

Primarily, I wish to express my gratitude to Minnesota State University, Mankato, an institution whose academic environment and values have significantly shaped both my intellectual and personal journey. The knowledge, opportunities, and experiences I gained here have deeply influenced my growth as a researcher and individual, and I am honored to have been part of this academic community.

Also, I extend my sincerest appreciation to my advisor, Dr. Rushit Dave, whose exceptional guidance, encouragement, and scholarly insight have been instrumental not only throughout the course of this thesis but also my entire academic journey at Minnesota State University. His unwavering support through every semester, his patient mentorship, and his constructive feedback have been vital in shaping my academic and research development. His dedication has inspired me to approach learning with endurance, curiosity, and confidence.

Most importantly, I am deeply indebted to my family and friends, whose love and support have been the foundation of this journey. To my family, thank you for your unwavering belief in me, your sacrifices, and your constant encouragement even in the most challenging times. Your faith has given me the strength to persist, and your love has been my greatest source of motivation. Especially to my parents, your presence has brought balance and joy to this demanding process, reminding me of the importance of support beyond academics.

This thesis is not solely a reflection of my individual effort but rather the result of the enduring love, strength, and encouragement of those who stood beside me. To my family, friends, and my mentor throughout all my semesters, I owe my deepest gratitude.

Glossary

- **Aspect-Based Sentiment Analysis (ABSA)**: A fine-grained sentiment analysis technique that identifies and categorizes opinions expressed about specific aspects or features of an entity.
- **Bi-LSTM (Bidirectional Long Short-Term Memory)**: A deep learning architecture that processes input sequences in both forward and backward directions to capture contextual dependencies in text or image data.
- **CNN (Convolutional Neural Network)**: A deep learning model particularly effective in feature extraction from images or text through convolutional layers that detect local patterns.
- **Cross-Validation (K-fold)**: A statistical technique used to evaluate the generalization ability of models by partitioning the dataset into k subset and training/testing iteratively on different folds.
- **Deep Learning**: A subset of machine learning that uses neural networks with multiple layers to learn complex representations of data automatically
- **DistilBERT**: A transformer-based deep learning model derived from BERT, optimized for faster training and inference while maintaining comparable performance. It is widely used for text-based and aspect-level classification.
- **F1-score**: A harmonic mean of precision and recall, providing a balance between the two metrics, especially useful when dealing with imbalanced datasets.
- **Machine Learning (ML)**: A field of artificial intelligence that uses algorithms to allow systems to learn patterns from data and make predictions or decisions without explicit programming.
- **Macro-F1**: An average of F1-scores computed independently for each class and then averaged, treating all classes equally regardless of their frequency.

- **Natural Language Processing (NLP)**: A branch of AI that enables machines to understand, interpret, and generate human language,
- **One-vs-Rest (OvR)**: A multi-class classification strategy that trains one classifier per class, distinguishing each class from all others.
- **Sentiment Analysis**: A text classification technique that determines the emotional tone (positive, neutral, or negative) expressed in textual content.
- **Support Vector Machine (SVM)**: A supervised machine learning algorithm used for classification and regression tasks, which finds the optimal hyperplane that best separates data points of different classes.
- **Support Vector Classifier (SVC)**: An implementation of SVM used for multi-class classification problems, often using kernel functions for non-linear decision boundaries.
- **Term Frequency-Inverse Document Frequency (TF-IDF)**: A numerical statistic used to reflect the importance of a word in a document relative to a corpus, commonly used for text feature extraction.
- **Tranformer Model**: A neural network architecture based on self-attention mechanisms, designed to process sequential data such as text efficiently.
- **XGBoost (Extreme Gradient Boosting)**: An ensemble machine learning algorithm that combines multiple week learners (decision trees) to improve prediction accuracy through boosting techniques.

Table of Contents

Table of Figures

Table of Tables

Abstract

Sentiment analysis has become a critical area of research in Natural Language Processing (NLP), enabling insights from unstructured text. Within this field, Aspect-Based Sentiment Analysis (ABSA) plays a practical role in domains such as healthcare, where patients drug reviews often contain diverse opinions across multiple aspects, including overall comments, perceived benefits, and side effects. However, aspect-level classification remains challenging due to class imbalance, subtle sentiment expression, and the limitations of traditional models.

This research investigates the performance of three modeling paradigms: traditional machine learning (SVM, SVC, and XGBoost), deep learning (CNN-BiLSTM), and transformer-based approaches (DistilBERT sentence-pair classification). Using the UCI Drug Review dataset, the study implements evaluation through stratified train/test splits, nested cross-validation, and multiple metrics including precision, recall, F1-score, and confusion matrices, to ensure robust comparison.

Results demonstrate that traditional models such as SVM and XGBoost offer interpretability and strong precision but struggle with recall on minority classes. The CNN-BiLSTM model improved contextual understanding but displayed weakness in neutral sentiment detection. DistilBERT, achieved the best overall performance, demonstrating higher F1-score and stronger balance across all aspects, effectively handling subtle sentiment distinctions.

This research provides a comprehensive multi-model comparison for ABSA in healthcare, highlighting trade-offs between efficiency, interpretability, and accuracy. It underscores the strength of transformer-based models for nuanced sentiment tasks while affirming the continued relevance of traditional approaches in practical applications.

Chapter 1

Introduction

In this digital age, as people increasingly rely on the internet to share their thoughts and opinions, sentiment analysis has become a crucial aspect of natural language processing (NLP). Social media platforms like Twitter, Instagram, Reddit, and Facebook serve not only as communication channels but also as valuable sources for under-standing public sentiment [1]. Whether businesses aim to refine their marketing strategies or researchers seek to detect fake news, extracting meaningful insights from textual data is essential [2]. As the amount of digital content rapidly expands, leveraging sentiment analysis enables us to interpret trends, distinguish genuine from misleading information, and make data-driven decisions [3].

To effectively analyze sentiments expressed in digital content, various machine learning and deep learning approaches have been developed. Sentiment analysis techniques determine the sentiment polarity of text – whether it is positive, negative, or neutral [4]. By utilizing natural language processing methods such as tokenization, word embeddings, and sentiment lexicons, these models can analyze a vast amount of textual data with high accuracy [5]. Businesses and organizations rely on sentiment analysis to gain valuable insights into customer feedback, brand perception, and market trends [6]. Advanced deep learning architectures such as convolutional neural networks (CNN) and long short-term memory networks (LSTM) further enhance the ability to capture contextual meaning and sentiment nuances. As sentiment analysis continues to evolve, it plays an increasingly important role in optimizing marketing strategies, improving customer experience, and detecting misinformation in digital content [7].

This paper reviews the advancements in sentiment analysis, evaluating different methodologies and their effectiveness across existing research studies. By comparing Machine

learning-based, deep learning-based and hybrid approaches, this study aims to provide insights into how different models can enhance decision-making, improve text classification accuracy, and optimize sentiment-based applications. Additionally, this study identifies limitations of different models, offering promising directions for future research.

The following sections provide an overview of sentiment analysis trends, a comprehensive literature review summarizing methodologies and applications of existing research. Finally, this paper discusses the challenges of current sentiment analysis research and concludes with insights into future directions.

1.1 Detecting Sentiment

Traditional sentiment analysis methods initially relied on lexicon-based or rule-based approaches that offered limited accuracy due to the complexity of natural language [1]. With the rise of machine learning, models such as Support Vector Machines (SVM) and Naïve Bayes (NB) became widely available, demonstrating improved precision and recall for text classification tasks [3]. These models leverage feature extraction techniques like Term Frequency-Inverse Document Frequency (TF-IDF), which have proven effective in capturing word frequency importance [16]. However, machine learning approaches often struggle with class imbalance and contextual dependencies, which are common in real-world data. Deep learning architectures, such as Convolutional Neural Networks (CNN) and Long Short-Term Memory networks (LSTM), have addressed these limitations by capturing sequential dependencies and contextual semantics [20]. More recently, transformer-based models like BERT have revolutionized sentiment detection by enabling bidirectional context modeling and subtle linguistic cue extraction [8]. These advancements have made aspect-level sentiment analysis (ABSA) feasible, where models can detect not only the overall polarity but also sentiment across multiple aspects within a single review [26].

1.2 Research Imperative

Detection of the aspect-level sentiment is crucial for various reasons, the most significant are listed below:

Granular Understanding of Feedback: In domains such as healthcare, e-commerce and financial services, users often express multiple sentiments within a single review. For example, patients may praise a drug's effectiveness while simultaneously criticizing its side effects. Traditional sentiment classifiers oversimplify such mixed opinions, while Aspect-Based Sentiment Analysis (ABSA)

provides a more precise and structured understanding of sentiment polarity at the aspect level [7, 6].

Enhanced Decision-Making for Organizations: Businesses and healthcare institutions increasingly depend on ABSA to interpret consumer opinions, refine products, and improve services. By identifying sentiment trends at the aspect level, organizations can focus on specific areas such as customer satisfaction, treatment outcomes, or service delivery to drive targeted improvement [12, 18].

Addressing Multi-Label Complexity: Many real-world reviews express multiple emotions or opinions simultaneously. Multi-label classification within ABSA enables the detection of overlapping sentiments across different aspects, offering a deeper understanding of contextual polarity [17, 19].

Managing Class Imbalance and Contextual Subtlety: ABSA models must address challenges such as skewed data distributions and subtle linguistic patterns that affect minority sentiment classes. Recent studies emphasize the need for models capable of handling these complexities through advanced feature extraction, oversampling, or hybrid learning approaches [3, 20, 25]

Advancing Research and Practical Applications: With the exponential growth of user-generated data, developing robust and scalable ABSA system is essential. These systems not only support sentiment-driven analytics in healthcare and business but also contribute to broader NLP advancements in contextual understanding and explainability [8, 26].

1.3 Objective

This study aims to evaluate the comparative effectiveness of traditional machine learning, deep learning, and transformer-based models in Aspect-Based Sentiment Analysis (ABSA),

focusing on their ability to detect sentiment across different aspects of healthcare-related drug reviews. Using the UCI Drug Reviews dataset, this research compares algorithms such as Support Vector Machines (SVM), XGBoost, CNN-BiLSTM hybrids, and DistillBERT to determine their capabilities in addressing class imbalance, contextual dependency, and multi-aspect polarity. The research endeavors to answer the following questions:

1. How do traditional machine learning models perform in comparison to deep learning and transformer-based approaches for aspect-level sentiment classification?
2. Which modeling framework demonstrates superior performance in accurately detecting sentiment across *Comments, Benefits,* and *Side Effects*?
3. To what extent do transformer-based architectures improve the handling of contextual subtleties and neutral sentiments compared to conventional algorithms?
4. What are the comparative strengths and weaknesses of each model in achieving higher interpretability, balance, and efficiency?

Research Question:

The main research question guiding this study is: "What is the comparative effectiveness of machine learning, deep learning, and transformer-based models in Aspect-Based Sentiment Analysis of healthcare-related reviews, and how can an optimized hybrid approach enhance sentiment classification accuracy, contextual understanding, and aspect-level interpretability?"

The objectives include identifying the best-performing algorithms for aspect-level sentiment detection, analyzing their robustness across multiple aspects, and conducting a comprehensive comparative study to highlight each model's limitations and potential for real-world applications in healthcare analytics.

Chapter 2

Literature Review

Over the past decade, sentiment analysis has evolved from simple text classification techniques to sophisticated machine learning and deep learning models. Early approaches primarily relied on lexicon-based and rule-based methods, which struggled to capture the nuances of human sentiment. However, the growing availability of large-scale labeled datasets enabled the adoption of supervised and unsupervised learning techniques, improving sentiment classification accuracy. Recent trends emphasize enhancing model performance for multilingual and cross-domain applications and improving the interpretability of AI-based sentiment classification. Studies have also explored various deep learning architectures, such as Bidirectional Encoder Representations from Transformers (BERT) models [8] and recursive deep models [9], to better understand sentiment in complex textual data. Additionally, sentiment analysis has been applied to a wide range of fields, including e-commerce [10] and social media, highlighting its versatility and impact on decision-making processes.

Building upon this foundation, the present study extends this body of research by focusing on Aspect-Based Sentiment Analysis (ABSA), a more granular approach that evaluates sentiment polarity for individual aspects within a single review. Through this synthesis, the literature review aims to situate current advancements within the broader historical development of sentiment analysis, demonstrating the transition from general polarity detection to aspect-level understanding across complex, domain-specific datasets.

2.1 Background

In order to comprehend the foundation of this study, it is essential to understand the principles of sentiment analysis, a key application area within Natural Language Processing (NLP)

and subfield of machine learning. Sentiment analysis focuses on interpreting and classifying subjective information in textual data to determine the sentiment polarity whether positive, negative, or neutral. Traditionally, machine learning algorithms have played a pivotal role in automating this process, allowing computers to analyze human language patterns through data-driven learning. Machine learning itself involves training computational models to recognize patterns, learn from data, and make predictions or classification on unseen information. Common paradigms include supervised learning, unsupervised learning, and reinforcement learning [16, 17, 18]. In supervised learning, algorithms are trained on labeled datasets where input data are paired with their correct output categories, enabling models to generalize patterns for new, unseen text. This research specifically employs supervised learning, as it involves training models on annotated datasets where sentiments are pre-labeled according to user opinions in drug reviews.

Building upon this foundation, deep learning has emerged as a transformative approach within sentiment analysis due to its ability to capture contextual and hierarchical language features. Deep learning models utilize artificial neural networks composed of multiple hidden layers that enable automatic feature extractions, thereby reducing the need for manual preprocessing. Architectures such as Convolutional Neural Networks (CNN) and Bidirectional Long Short-Term Memory (BiLSTM) networks have proven effective in understanding semantic relationships and sequential dependencies within textual data. More recently, transformer-based models including Bidirectional Encoder Representations from Transformers (BERT) and its lightweight variant DistilBERT have revolutionized sentiment analysis through their self-attention mechanism, enabling a deeper understanding of contextual nuances across long text spans. [8, 9, 10]. In the context of this thesis, all three methodological families – traditional, deep learning, and transformer base – are comparatively examined to determine their effectiveness in Aspect-Based

Sentiment Analysis (ABSA), where the goal is to evaluate specific aspects of healthcare-related reviews such as "*Benefits*", "*Side Effects*" and "*Comments*".

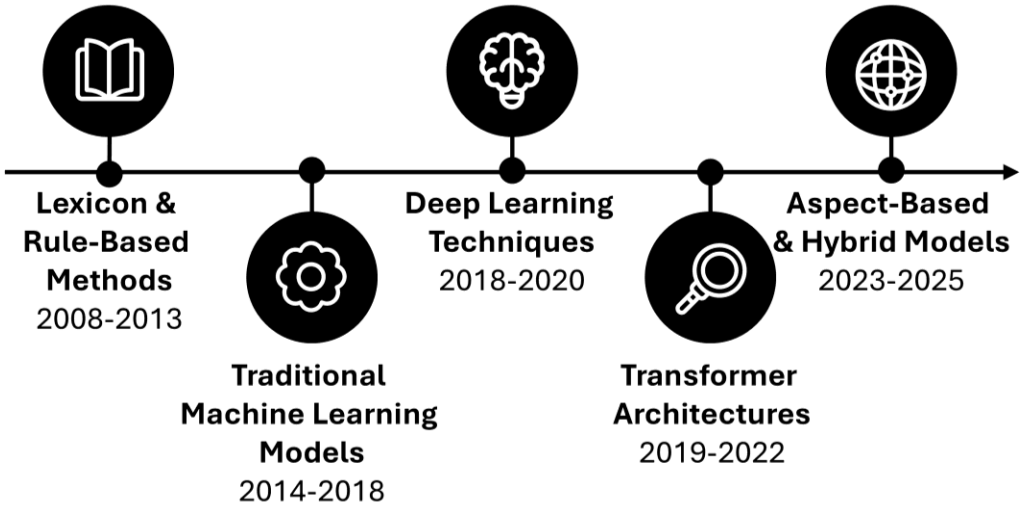


Figure 1. Evolution of Sentiment Analysis Approach (2008-2025)

2.2 Search Strategy

To identify and review the most relevant studies for this research, comprehensive and systematic search was conducted across major academic databases including IEEE Xplore, ACM Digital Library, ScienceDirect, and Google Scholar. Additional materials were accessed through the Minnesota State University Library, ensuring that only credible sources were considered. The research strategy employed key terms such as "sentiment analysis," "machine learning," "deep learning," "transformer models," "e-commerce," "natural language processing," and combinations thereof using Boolean operators.

To maintain both quality and contemporary relevance, the search was limited to studies published between 2017 and 2024. The inclusion criteria prioritized peer-reviewed articles, conference proceedings, and journal publications that specifically addressed sentiment analysis applications in e-commerce contexts using machine learning approaches, while excluding studies

focused solely on theoretical frameworks without empirical validation. This systematic approach ensured that the selected body of literature provided a balanced and up-to-date foundation for comparing traditional, deep learning, and transformer-based methods in Aspect-Based Sentiment Analysis, especially in the context of the UCI Drug Reviews dataset used in this research.

2.3 Previous Methods

Sentiment analysis has become a crucial tool for understanding customer opinions and enhancing e-commerce platforms. Shathik et al. [11] provides a comprehensive review of sentiment analysis applications using machine learning techniques, discussing key challenges and improvements. A significant contribution is the categorization of machine learning based sentimental analysis methods, emphasizing their applications, strengths, and limitations. Chaturvedi et al. [12] examine sentiment analysis in business intelligence, emphasizing its role in decision-making. The paper introduces sentiment analysis as a key tool to process unstructured data, offering practical and computational insights into its application for improving decision-making processes in competitive business environments. More recently, Huang et al. [13] reviewed current machine learning techniques for sentiment analysis in e-commerce, highlighting their effectiveness. The study systematically reviewed 54 experimental papers on sentiment analysis in e-commerce, results show that Support Vector Machine (SVM) and Naïve Bayes (NB) were the most widely used machine learning models consistently shown better results than others, whereas Recurrent Neural Networks (RNN) and its variation like Long Short-Term Memory (LSTM) and Gated Recurrent Units (GRU) were dominant in deep learning models. The study also finds out that deep learning techniques like BERT, Convolutional Neural Networks (CNN), RNN, and LSTM classification are frequently used in recent studies. Alonso et al. [14] explore sentiment analysis for fake news detection, comparing models like Random Forest (RF), NB, SVM, LSTM,

and CNN. It discusses how fake news often contains emotionally charged language to manipulate readers and increase engagement. Various sentiment analysis techniques are evaluated, and the result shows that hybrid models outperform standalone classifiers. Kawade et al. [15] apply sentiment analysis to Uri attack using R for textual data processing. It highlights how sentiment analysis can mine public emotions from social media platforms like Twitter and process them for better understanding of public opinions on events. As shown in Figure 2, the sentiment analysis process generally involves stages such as data collection, preprocessing, feature extraction, model training, and evaluation.

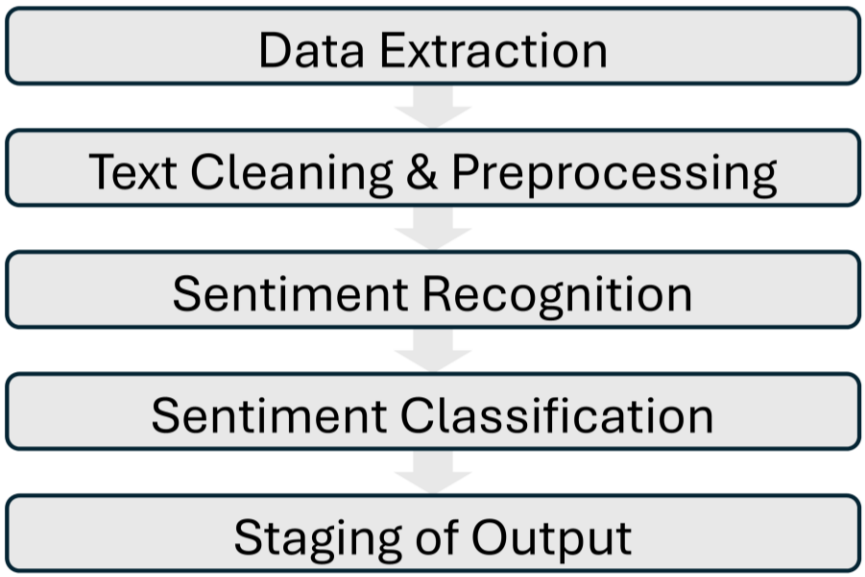


Figure 2. General Sentiment Analysis Process Pipeline

2.3.1 Machine Learning Approaches for Sentiment Analysis

Building on the foundational overview of sentiment analysis techniques, several studies have implemented traditional machine learning techniques that have been widely applied for sentiment analysis in e-commerce. Yi et al. [16] apply multinomial NB, K-Nearest Neighbors (KNN), and SVM combined with collaborative filtering to recommend products and shops based on customer reviews, achieving improved recommendation accuracy measured by Mean Absolute

Error (MAE) and Root Mean Square Error (RMSE). By integrating collaborative filtering and product-product similarity techniques, the study provides a framework for identifying customer preferences, enhancing e-commerce experiences. Deniz et al. [17] introduce a multi-label classification approach using Binary Relevance, Multi-label KNN, and One-vs-Rest classifiers with Logistic Regression (LR), Stochastic Gradient Descent (SGD), and XGBoost. The best performing model was One-vs-Rest-XGBoost with 94.2% accuracy and 0.89 F1-score, indicating a low misclassification rate. Among feature extraction techniques, Term Frequency-Inverse Document Frequency (TF-IDF) performed better than word embeddings and transformers. The Saura et al. [18] utilize SVM and Latent Dirichlet Allocation (LDA) to extract key indicators for startup business success, evaluated through topic coherence scores and classification accuracy. LDA identifies words and separates each word into a different document, randomly identifies distribution of the topics in a sample and then selects the main topics found in the sample. The identified topics are further analyzed using Nvivo software to extract meaningful insight. Alnasrawi et al. [19] enhance sentiment classification by integrating text network features with various machine learning models, including SVM, RF, LR, NB, Decision Trees (DT), Neural Networks (NN), Gradient Boosting (GB), and AdaBoost. Among all methods, NN outperformed other models with 88.7% accuracy and 0.85 F1-score, and incorporating network-based features improved classification performance by 5-8% across precision, recall and F1-score metrics, particularly for NN and ensemble methods.

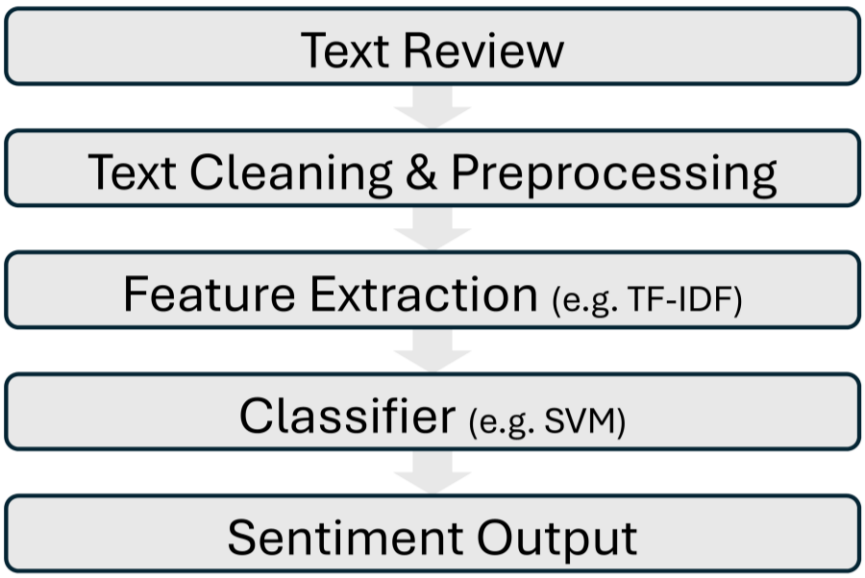


Figure 3. Example of Machine Learning Workflow

2.3.2 Deep Learning Approaches for Sentiment Analysis

Despite the effectiveness of traditional machine learning models, their limitations in handling complex linguistic patterns have led researchers to explore deep learning techniques for more nuanced sentiment classification. Deep learning techniques have been widely adopted for sentiment analysis due to their ability to capture complex language patterns. Mehta et al. [20] propose a CNN based sentiment classification model enhanced with Word2Vec and TF-IDF to analyze customer reviews in the apparel sector using deep learning models, achieving 92.3% accuracy and 0.91 F1-score. The study does not evaluate other deep learning models, which could improve sentiment classification. Lin [21] employs Natural Language Processing (NLP) methods to analyze correlation between e-commerce customer review features and product recommendations. The researcher examined several different models like Lasso Regression, Ridge Regression, Linear Kernel SVM, Radial Basis Function (RBF) Kernel SVM, RF, XGBoost, and LightGBM. The best result was achieved by LightGBM with 0.847 R-squared value and the correlation between review ratings and product recommendations was found to be strong. Kundra

et al. [22] compares traditional machine learning models such as NB, SVM, RF, and AdaBoost with deep learning based RNN and GRU. The research results show that the best performing model is RNN-GRU with 89.5% accuracy and 0.87 F1-score for unstructured data, but SVM achieved 91.2% accuracy for structured data analysis and performed best in both structured and unstructured data analysis, demonstrating that RNN captures the nuances of unstructured and con-text-rich information while traditional models excel with structured inputs.

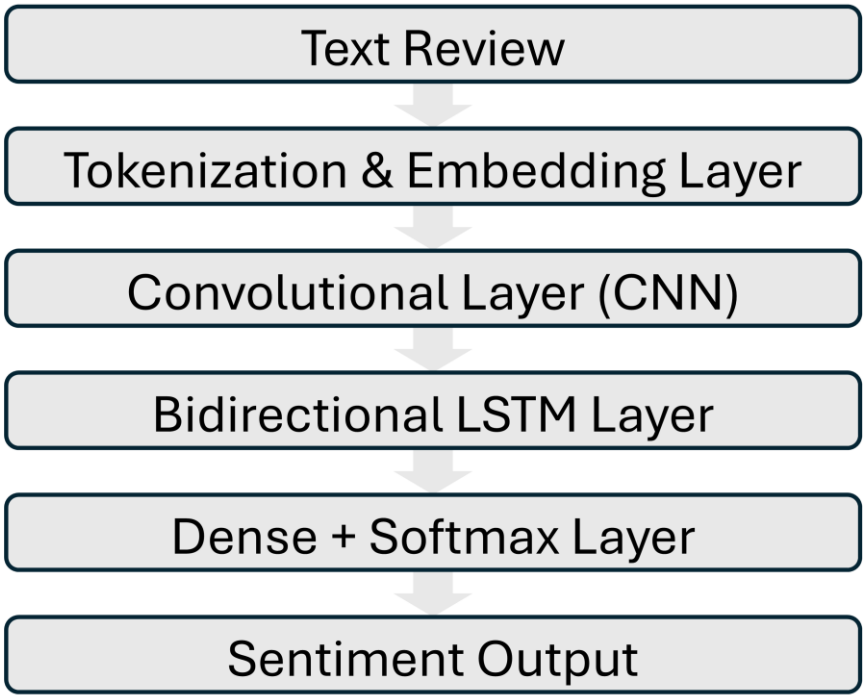


Figure 4. Example of Deep Learning Workflow

2.3.3 Hybrid Approaches for Sentiment Analysis

To capitalize on the strength of both traditional and deep learning approaches, recent studies have proposed hybrid models that aim to achieve higher accuracy and adaptability in sentiment analysis. Hybrid models have been developed to enhance sentiment analysis accuracy by combining traditional machine learning and deep learning methods. Zhao et al. [23] introduced new optimized machine learning algorithm called the Local Search Improvised Bat Algorithm-

based Elman Neural Network (LSIBA-ENN) for sentiment analysis, a new term weighting technique called Log Term Frequency-based Modified Inverse Class Frequency (LTF-MICF) for sentiment classification, and a Hybrid Mutation-based Earthworm Algorithm (HMEWA) for feature selection. The LSIBA-ENN model achieved 95.2% accuracy and 0.94 F1-score, outperforming traditional classifiers, and the LTF-MICF weighting scheme improved performance by 3-5% compared to TF-IDF and Word2Vec-based approaches. There are several studies combining the CNN and LSTM methods to improve the accuracy of sentiment analysis. Vatambeti et al. [24] employs a ConvBiLSTM model, integrating CNN and (Bi-LSTM) with an Elephant Herd Optimization algorithm (EHO). EHO is a swarm-based optimization method used to optimize the weight of the Bi-LSTM, deep learning model parameters. The Optimized model achieved 93.8% accuracy with 0.92 precision and 0.91 recall on Twitter food service data. Behera et al. [25] present Co-LSTM, a combination of CNN and LSTM, for sentiment classification in social media data. The proposed model achieved 94.5% accuracy and 0.93 F1-score, outperforming standalone CNN with 87.2% accuracy and LSTM 89.7% accuracy. The model is highly adaptable in examining big social data and is free from any particular domain, unlike conventional machine learning approaches. Co-LSTM performed particularly well on long and complex texts by effectively capturing dependencies between words. Ray et al. [26] also integrated CNN with rule-based methods to improve aspect-level sentiment analysis. The model achieved 88.4% accuracy and 0.86 F1-score, incorporating hierarchical clustering and dependency parsing to refine aspect categorization and improve accuracy. Jain et al. [27] implemented a CNN-LSTM model for consumer sentiment analysis, and the model demonstrated 91.7% accuracy with 0.89 precision and 0.88 recall, higher than standalone deep learning and machine learning models. Furthermore, Priyadarshini et al. [28] develop a hybrid LSTM-CNN model with Grid Search Optimization. The

model achieved the highest performance with 96.1% accuracy, 0.95 precision, 0.94 recall, and 0.94 F1-score. The model is compared with baseline algorithms including CNN (88.2% accuracy), KNN (82.1% accuracy), LSTM (90.3% accuracy), CNN-LSTM (92.8% accuracy) and LSTM-CNN (93.4% accuracy), and LSTM-CNN-Grid Search achieved higher performance as the research incorporated grid search as a hyperparameter optimization technique to minimize pre-defined losses.

As illustrated in Figure 5, the machine learning workflow for sentiment analysis often integrates various stages, including data input, feature extraction, model selection, and performance evaluation.

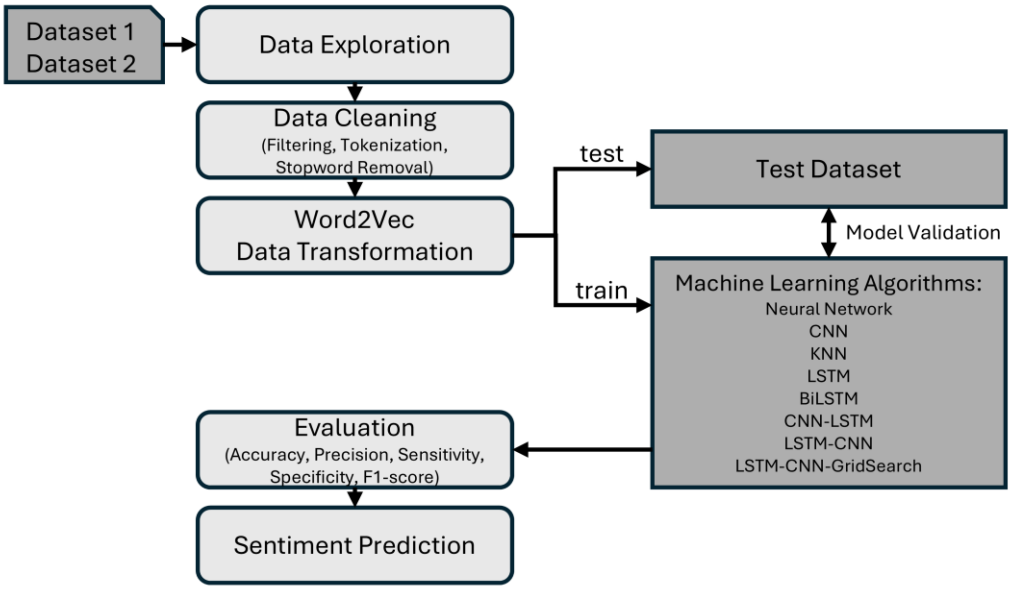


Figure 5. Example of Sentiment Analysis Model Development Workflow

2.3.4 Real-time Application

As sentiment analysis techniques continue to evolve, there is growing interest in applying these models in real-time systems to provide immediate feedback and actionable insights. Several studies focus on real-time sentiment analysis and AI-driven emotion visualization. Jabbar et al. [29] implement a SVM based real-time sentiment analysis system within a mobile application using Python and Flask. The system achieved 87.3% accuracy with an average response time of 0.8 seconds per query. When users submit a review, the application processes the text through the model, which predicts the sentiment polarity and provides immediate feedback. Li et al. [30] propose an explanation framework for AI-driven sentiment analysis, leveraging Bi-LSTM with attention mechanisms to visualize emotion-triggering words. The proposed framework achieved 91.2% accuracy and 0.88 F1-score while providing 78% user satisfaction in explainability surveys. The framework emphasizes considering the cause and trigger of emotions as the explanation for the deep learning-based emotion analysis. The proposed method has more functionality, usability, reliability, and performance compared to SHapley Additive exPlanations (SHAP), which is a model interpretation package developed by Python and Local Interpretable Model-agnostic Explanations (LIME) which is an explainable framework for black box deep learning models using a local explainable model to explain each separate prediction, with 15% higher user comprehension scores and 20% faster explanation generation time.

These advancements underscore how real-time, explainable ABSA systems can enhance user experience, interactivity, and interpretability by providing both sentiment polarity and context-specific explanations across multiple aspects.

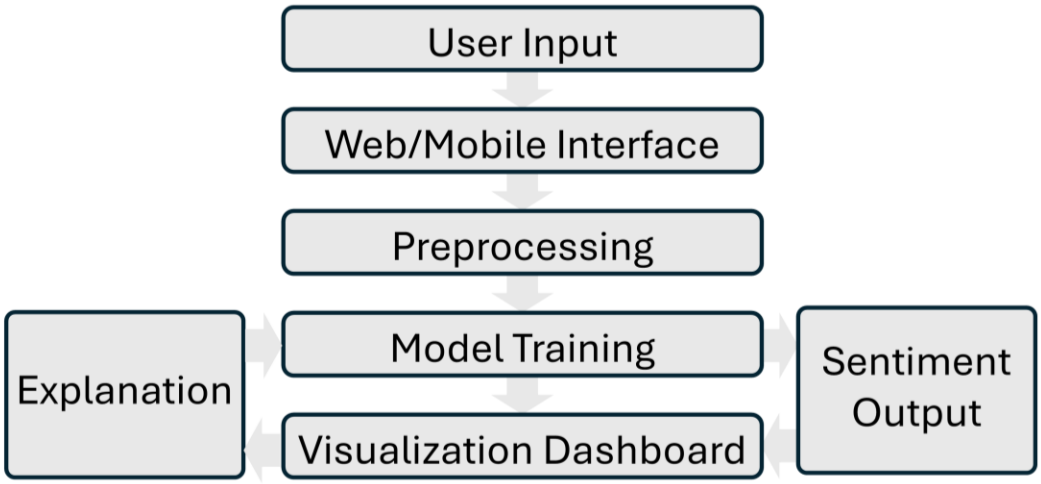


Figure 6. Real-time and Explainable Sentiment Analysis System Architecture

2.3.5 Aspect-Based Sentiment analysis

To further advance sentiment analysis research, recent studies have introduced novel Aspect-Based Sentiment Analysis (ABSA) models that emphasize contextual understanding, interpretability, and real-time adaptability. These approaches move beyond sentence-level sentiment classification and docus on identifying the sentiment polarity of specific aspects within a review, such as product quality, price, or user experience. Tian et al. [31] propose a new Context Denoising Attention (CDA) model to address the problem of noisy contextual information surrounding aspect terms. The CDA framework utilizes an adaptive gating mechanism within a transformer encoder to selectively filter out irrelevant tokens, allowing the model to focus only on aspect-relevant sentiment expressions. Experimental evaluations conducted on benchmark datasets such as SemEval2014 (Restaurant and Laptop) and MAMS demonstrate significant improvement, achieving up to 4.7% higher F1-score compared to baseline transformer models. The study highlights the importance of eliminating contextual noise to enhance both accuracy and interpretability in aspect-level sentiment prediction.

While Tian et al. [31] focused on refining aspect-level precision by filtering contextual noise, other researchers have aimed to enhance sentiment understanding across longer and more complex textual structures. Yan et al. [32] present the Document-level Aspect Relation Transformer (DART)., a hierarchical transformer-based model designed to analyze long-form reviews where aspects and sentiment expressions are distributed across multiple sentences. The DART framework integrates both local sentence-level and global document-level attention layers to effectively capture inter-sentence dependencies. This hierarchical structure allows the model to recognize sentiment relationships between distant aspects and their corresponding opinions. The model outperforms traditional transformer-based architectures such as BERT and RoBERTa, achieving over 3% improvement in macro-F1 across multiple ABSA datasets. The research demonstrates that hierarchical and relational modeling is essential for comprehensive sentiment interpretation in domains with lengthy user feedback, including e-commerce and healthcare reviews.

To enhance data robustness and generalization, Huang et al. [33] introduce a Linking Word-Guided Multidimensional Emotional Data Augmentation and Adversarial Contrastive Training framework for ABSA. This approach leverages linguistic connectors such as “however”, “but”, and “although” to generate semantically coherent augmented samples that preserve aspect-specific sentiment orientation. The model incorporates adversarial contrastive learning to maintain sentiment consistency under textual perturbations, effectively reducing overfitting and improving resilience. Experimental results across five benchmark datasets reveal a 5.6% improvement in macro-F1 compared to conventional BERT and GRU models. The study emphasizes the role of linguistic-driven augmentation and contrastive learning in enhancing ABSA performance, particularly in low-resource or imbalanced datasets.

Furthermore, Aziz et al. [34] propose a Unified Multi-Task BERT framework for Aspect-Based Sentiment Analysis, which integrates multiple ABSA subtasks including aspect extraction, opinion detection, and sentiment polarity classification, into a single architecture. Unlike traditional pipelines that require separate training for each task, this unified model employs shared transformer layers and task-specific attention heads to simultaneously learn relationships across subtasks. The system achieves an overall 2~3% higher F1-score than single task BERT models while reducing training time and computational complexity. This unified approach promotes model efficiency and domain adaptability, supporting cross-domain transfer learning where ABSA can be applied to various contexts such as product reviews, hotel feedback, and medical evaluations.

These recent studies from 2024 to 2025 collectively illustrate the continuous evolution of ABSA toward context-aware, explainable, and real-time systems. The integration of context denoising mechanisms [31], hierarchical document modeling [32], linguistic-based data augmentation [33] and unified multi-task learning [34] demonstrates the field's shift from static sentiment analysis to intelligent, adaptive architectures capable of understanding fine-grained user opinions. These approaches not only improve classification accuracy and F1-score but also strengthen interpretability and robustness, paving the way for ABSA models that can operate effectively in real-world situations, real-time environments where instant and aspect-specific insights are essential for decision-making and customer experience enhancement.

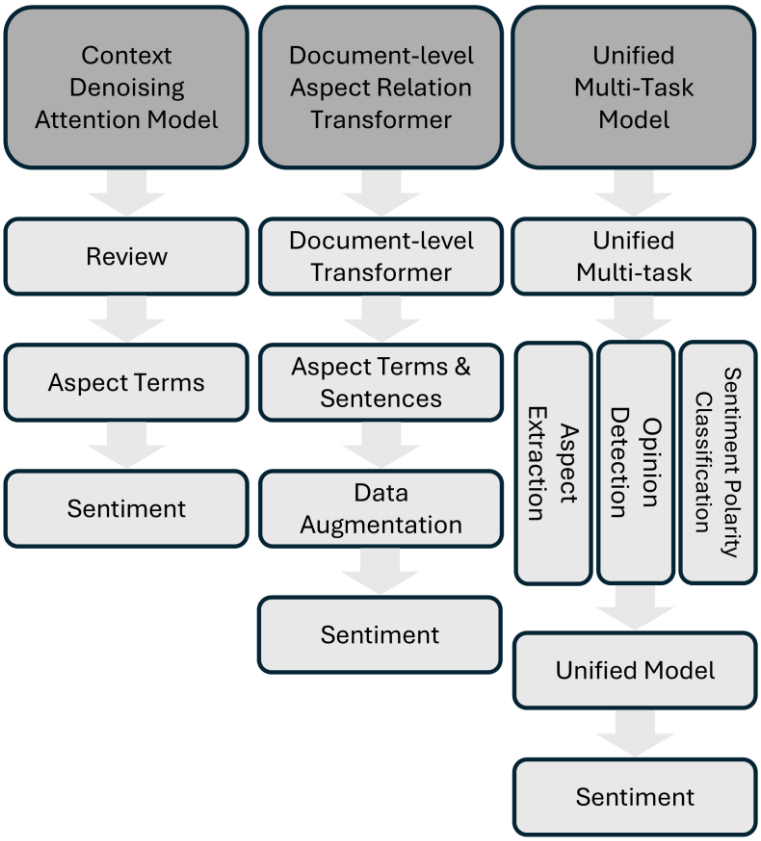


Figure 7. Aspect-Based Sentiment Analysis (ABSA) model Comparison

2.3.6 Literature Analysis

The presented Table 1 provides a comprehensive overview of existing literature in sentiment analysis, highlighting the diversity of datasets, model types, domains, and performance metrics employed in recent research. Each study represents a distinct methodological approach aimed at improving sentiment classification accuracy across various contexts, ranging from general sentiment detection to specialized applications such as e-commerce, business intelligence, social media, and healthcare. The majority of studies utilize accuracy and f1-score as the primary evaluation metrics, underscoring their importance in assessing model robustness, particularly when dealing with imbalanced datasets that include varying proportions of positive, neutral and negative sentiments.

From a methodological perspective, the studies listed reveal the progressive evolution of sentiment analysis from traditional machine learning (ML) approaches toward deep learning (DL) and hybrid architectures. Early research such as Shathik et al. [11], Chaturvedi et al. [12], and Huang et al. [13] provided comprehensive reviews of existing techniques, establishing the foundational role of ML models like Support Vector Machines (SVM) and Naïve Bayes (NB), which achieved consistent accuracy across diverse datasets. Subsequent works such as Deniz et al. [17], Yi et al. [16], and Saura et al. [18] expanded this framework by integrating multi-label classification, topic modeling (LDA), and collaborative filtering methods to enhance contextual understanding and recommendation accuracy. The incorporation of ensemble and boosting techniques – for example XGBoost in Deniz et al. [17] – demonstrated significant improvements, reaching 94.2% accuracy and F1-score of 0.89, suggesting that boosted tree-based algorithms are particularly effective for structured sentiment data.

As the field matured, deep learning and hybrid models gained prominence for their ability to capture linguistic complexity and contextual dependencies. Studies such as Mehta et al. [20] and Kundra et al. [22] employed CNN, LSTM, and RNN-GRU architectures, achieving accuracy above 90%, highlighting the strength of sequential and convolutional models in handling unstructured text. Moreover, hybrid frameworks such as the LSIBA-ENN by Zhao et al. [23] and Co-LSTM by Behera et al. [25] illustrated that combining ML and DL paradigms can make superior performance, often surpassing single-model baselines. The highest-performing model identified across these studies was LSTM-CNN with Grid Search Optimization (Priyadarshini et al. [28]), which achieved 96.1% accuracy and an F1-score of 0.94, emphasizing the role of hyperparameter tuning in optimizing deep learning architectures.

In addition to the conventional sentiment classification, some researchers explored aspect-level and real-time applications. Ray et al. [26] introduced a hybrid CNN and rule-based model for aspect-level sentiment analysis, obtaining 88.4% accuracy, while Jabbar et al. [29] implemented a real-time SVM-based framework achieving 87.3% accuracy with rapid response times, showcasing practical applicability. Furthermore, Li et al. [30] introduced explainable AI through Bi-LSTM with attention, enhancing both accuracy of 91.2% and user satisfaction of 78%, marking a significant step toward interpretable and user-friendly sentiment models.

More recent advancements, as reflected in Aspect-Based Sentiment Analysis (ABSA) studies [31-34], show a paradigm shift toward context-aware, explainable, and real-time sentiment models. Tian et al. [31] introduced Context Denoising Attention (CDA) model to filter irrelevant tokens in transformer encoders, improving F1-score by 4.7% on benchmark datasets such as SemEval2014. Yan et al. [32] extended this by proposing DART, a hierarchical transformer model capable of linking aspect-level and document-level dependencies, resulting in over 3% improvement in macro F1. Meanwhile, Huang et al. [33] utilized linguistic cues such as “however” and “although” to design an emotional data augmentation and adversarial contrastive training framework, achieving a 5.6% performance gain in macro-F1. Aziz et al. [34] contributed the Unified Multi-Task BERT architecture that integrates aspect extraction, opinion detection, and sentiment classification into a single system, achieving 2~3% higher F1-scores while reducing training complexity. These developments illustrate the transition from static, single-sentence sentiment classification toward adaptive, multi-task, and linguistically guided ABSA systems that enhance interpretability, robustness, and domain adaptability.

Overall, the collective findings from Table 1 illustrate the trajectory of sentiment analysis research from statistical machine learning toward deep, hybrid and attention-based architectures.

The trend clearly indicates that hybrid and transformer-driven methods are achieving superior results due to their capability to model contextual semantics and manage complex text structures. These insights reinforce the motivation for this study – to compare the performance and interpretability of traditional, deep learning, and transformer-based approaches in Aspect-Based Sentiment Analysis (ABSA) within the healthcare domain using the UCI Drug Reviews datasets, extending prior comparative research into a more domain-specific and fine-grained application.

Table 1. Literature Analysis

Title	Dataset	Model Type	Best Model	Best Metric
A Literature Review on Application of Sentiment Analysis Using Machine Learning Techniques [11]	Multiple datasets	ML/DL Review		
Sentiment Analysis using Machine Learning for Business Intelligence [12]	Business data	ML		
Sentiment Analysis in E-Commerce Platforms: A Review of Current Techniques and Future Direction [13]	54 studies meta-analysis	ML/DL Review	SVM, NB, LSTM, GRU	Accuracy / F1-score
Sentiment Analysis for Fake News Detection [14]	Fake news datasets	ML/DL/Hybrid	Hybrid models	Accuracy / Precision / Recall / F1-score
Sentiment Analysis: Machine Learning Approach [15]	Twitter data (Uri attack)	ML		
Machine learning based customer sentiment analysis for recommending shoppers, shops based on customers' review [16]	Customer reviews	ML + Collaborative Filtering	Multinomial NB + Collaborative Filtering	MAE, RMSE improvement
Multi-Label Classification of E-Commerce Customer Reviews via Machine Learning [17]	E-commerce customer reviews	Multi-label ML	One-vs-Rest-XGBoost	Accuracy: 94.2% F1-score: 0.89
Detecting indicators for startup business success: Sentiment analysis using text data mining [18]	Startup business data	SVM + LDA	SVM + LDA	Topic coherence & classification accuracy
Improving sentiment analysis using text network features within different machine learning algorithms [19]	Text network data	ML Ensemble	Neural Networks	Accuracy: 88.7% F1-score: 0.85
Sentiment Analysis on E-Commerce Apparels using Convolutional Neural Network [20]	Apparel sector reviews	CNN + Word2Vec/TF-IDF	CNN + Word2Vec/TF-IDF	Accuracy: 92.3% F1-score: 0.91

Sentiment Analysis of E-commerce Customer Reviews Based on Natural Language Processing [21]	E-commerce reviews	ML/Regression	LightGBM	R-Squared: 0.847
Digital Emotions using Sentiment Analysis for Predictive Insights on Customer Recommendations [22]	Structured / Unstructured data	ML/DL	RNN-GRU (un-structured) / SVM (structured)	RNN-GRU Accuracy: 89.5% F1-score: 0.87 SVM Accuracy: 91.2%
A machine learning-based sentiment analysis of online product reviews with a novel term weighting and feature selection approach [23]	Product reviews	Hybrid	LSIBA-ENN + LTF-MICF	Accuracy: 95.2% F1-score: 0.94
Twitter sentiment analysis on online food services based on elephant herd optimization with hybrid deep learning technique [24]	Twitter food service data	ConvBiLSTM + EHO	ConvBiLSTM + EHO	Accuracy: 93.8% Precision: 0.92 Recall: 0.91
Co-LSTM: Convolutional LSTM model for sentiment analysis in social big data [25]	Social media data	Co-LSTM	Co-LSTM	Accuracy: 94.5% F1-score: 0.93
A Mixed approach of Deep Learning method and Rule-Based method to improve Aspect Level Sentiment Analysis [26]	Aspect-level data	CNN + Rule-based	CNN + Rule-based	Accuracy: 88.4% F1-score: 0.86
A Hybrid CNN-LSTM: A Deep Learning Approach for Consumer Sentiment Analysis Using Qualitative User-Generated Contents [27]	Consumer data	CNN-LSTM	CNN-LSTM	Accuracy: 91.7% Precision: 0.89 Recall: 0.88
A novel LSTM–CNN–grid search-based deep neural network for sentiment analysis [28]	General sentiment data	LSTM-CNN + Grid Search	LSTM-CNN + Grid Search	Accuracy: 96.1% Precision: 0.95 Recall: 0.94 F1-score: 0.94
Real-time Sentiment Analysis On E-Commerce Application [29]	Mobile app reviews	SVM	SVM	Accuracy: 87.3% Response time: 0.8s
An explanation framework and method for AI-based text emotion analysis and visualisation [30]	Emotion analysis data	Bi-LSTM + Attention	Bi-LSTM + Attention	Accuracy: 91.2% F1-score: 0.88 User Satisfaction: 78%

Aspect-based Sentiment Analysis with Context Denoising [31]	SemEval2014, MAMS	Transformer	CDA model	4.7% improvement in F1-score
Modeling Complex Interactions in Long Documents for Aspect-Based Sentiment Analysis [32]	SemEval2014, multi-domain ABSA datasets	Hierarchical Transformer	DART (Document-level Aspect Relation Transformer)	3% improvement in macro-F1
Enhancing aspect-based sentiment analysis with linking words-guided emotional augmentation and hybrid learning [33]	Five Benchmark ABSA datasets	Data Augmentation + Adversarial Training	World-Guided Emotional Contrastive Model	5.6% improvement in macro-F1
Unifying aspect-based sentiment analysis BERT and multi-layered graph convolutional networks for comprehensive sentiment dissection [34]	Multi-domain ABSA	Unified Multi-Task BERT	Unified Multi-Task BERT	2~3% higher F1-score than single-task BERT models

Chapter 3

Dataset

The dataset used in this research is the UCI Drug Reviews Dataset, obtained from the University of California, Irvine (UCI) Machine Learning Repository. This dataset serves as a valuable resource for analyzing patient feedback on various prescriptions and over-the-counter medications. Unlike general-purpose sentiment datasets found in e-commerce or social media context, the UCI Drug Reviews dataset provides detailed, user-generated reviews accompanied by corresponding ratings and medical conditions, making it highly suitable for Aspect-Based Sentiment Analysis (ABSA). The dataset comprises approximately 215,063 drug reviews, encompassing over 900 different medications, spanning multiple drugs and treatment categories, each review labeled with attributes such as drug name, condition, review text, and rating. The diversity of entries allows for a comprehensive evaluation of public sentiment within the healthcare domain, providing an authentic representation of patient experiences, treatment effectiveness, and side effects.

The dataset was selected not only for its size but also for its multidimensional nature, which aligns with the objective of this research. In healthcare-related reviews, users often express differing opinions about various aspects of a single drug reporting positive experiences regarding effectiveness while simultaneously highlighting negative reactions or discomfort. To address this complexity, the dataset was refined and preprocessed to focus on three primary aspects: Comments, Benefits, and Side Effects, which collectively capture the multidimensional nature of healthcare-related sentiment. The “Comments” aspect represents general impressions and user experiences, while “Benefits” highlight therapeutic outcomes or perceived effectiveness, and

"Side Effects" reflect adverse reactions or medical concerns. This structure enables the ABSA framework to model sentiment at the aspect level, rather than simply determining the overall polarity of the review.

Table 2 Example of Dataset

Drug Name	Condition	Original Review Text	Aspect	Aspect Text	Mapped Rating	Sentiment Label
Zoloft	Depression	"Zoloft helped me feel calmer and more in control, but I had headaches for the first week"	Comments	"helped me feel calmer and more in control"	8	Positive
			Benefits	"helped me feel calmer and more in control"	8	Positive
			Side Effects	"had headaches for the first week"	3	Negative

Each aspect was paired with a corresponding sentiment label – positive, neutral, or negative – mapped from the original rating scores. Ratings from 1-3 were categorized as negative, 4-7 as neutral, and 8-10 as positive. This mapping ensured balanced sentiment representation and allowed for cross-comparison between machine learning, deep learning, and transformer-based models. During preprocessing, several text cleaning and normalization procedures were applied such as lowercasing, punctuation removal, and stop-word filtering to ensure data consistency and reduce noise.

Given the dataset's large size and variability, maintaining quality control was critical. A secondary validation step was performed to ensure data integrity by cross-checking for duplicate reviews and mismatched ratings. Additionally, the dataset exhibited a class imbalance problem,

with positive sentiments dominating across all aspects which is an expected trend in medical feedback datasets, where satisfied users are more likely to leave reviews. To mitigate this imbalance, stratified sampling was used during train-testing splitting, and K-fold cross-validation (5 outer folds, 3 inner folds) was implemented to ensure robust performance evaluation. Data augmentation techniques, such as synonym replacement and back translation were explored for minority classes to improve model generalization in neutral and negative sentiment categories.

The feature extraction process differed depending on the model family. For traditional machine learning models such as Support Vector Machines, Support Vector Classifier, and XGBoost, the text data were vectorized using Term Frequency-Inverse Document Frequency (TF-IDF) with both word-level and character-level n-grams. These features were stored as sparse matrices for efficiency. In contrast, deep learning models like CNN-BiLSTM utilized tokenized sequences converted into integer indices through Keras' Tokenizer function, with embedding layers trained to learn contextual representations. Transformer-based models, including DistilBERT, required input formatting as paired sequences:

$$[CLS]\ review\ text\ [SEP]\ aspect\ name\ [SEP]$$

This structure allowed the transformer to contextualize aspect information alongside the review text, thereby enhancing interpretability and sentiment precision.

After preprocessing, each aspect-specific dataset contained approximately 4,000 to 5,000 balanced entries, which were used for comparative model evaluation. Figure 2 illustrates the distribution of sentiment classes across all three aspects revealing that positive reviews dominate across "Benefits" and "Side effects" while "Comments" exhibit a more balanced spread. This

pattern underscores the dataset's inherent complexity and the need for models capable of handling multi-aspect dependencies and imbalanced sentiment representation.

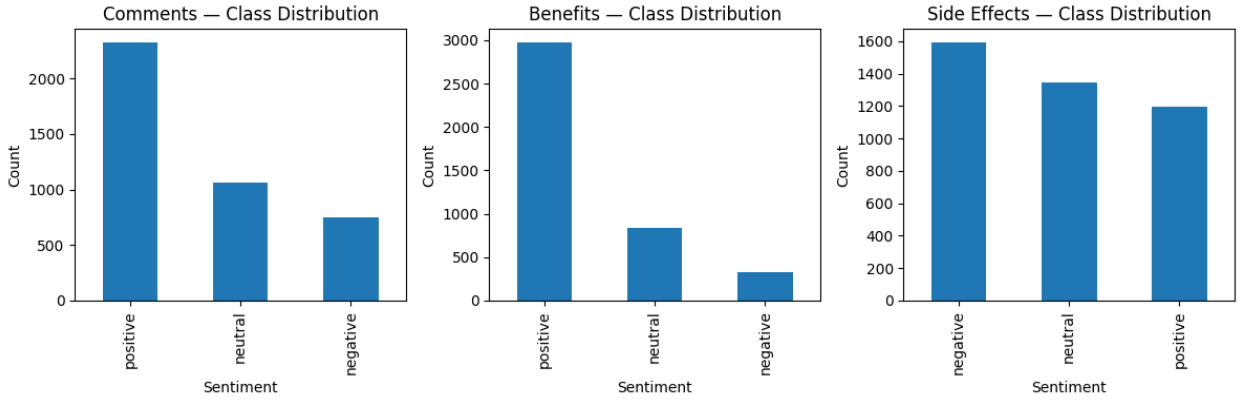


Figure 8. Class Distribution of Dataset

Chapter 4

Methodology

This section demonstrates the systematic approach adopted to develop and evaluate machine learning, deep learning, and transformer-based models for Aspect-Based Sentiment Analysis (ABSA) using UCI Drug Reviews dataset. The methodology encompasses dataset preprocessing, feature extraction, and multi-model experimentation to classify sentiment across the distinct aspects – *Comments, Benefits*, and *Side Effects*. Initially, text data were cleaned and transformed into structured feature representations using TF-IDF for traditional models and token embeddings for deep learning and transformer architectures. Subsequently, multiple algorithms including SVM, SVC, XGBoost, CNN-BiLSTM, and DitillBERT were implemented and optimized to identify the most effective model for multi-class sentiment prediction. The section also details the validation framework involving stratified train-test splits, K-fold cross-validation, and performance evaluation through precision, recall, F1-score, and accuracy metrics to ensure the robustness and reproducibility of the results.

4.1 Computational and Software Used

The computational resources utilized for this study were provided by Minnesota State University, Mankato computing lab, with an Intel Core i7-7700K CPU, 32GB of RAM, and NVIDIA GeForce GTX 980 GPUs with 4GB of memory. These configurations ensured adequate capacity for training both traditional machine learning models and computationally intensive deep learning architecture. Model implementation and experimentation were primarily conducted in Google Colab, with offered GPU acceleration for efficient model execution and training, complemented by local testing environments using Jupyter Notebook and Visual Studio Code for debugging and data preprocessing tasks.

Additionally, for software, Python 3.10 served as the main programming environment. Core libraries included scikit-learn for machine learning models such as SVM, SVC, and XGBoost; TensorFlow and Keras for implementing deep learning architecture such as CNN-BiLSTM; and Hugging Face Transformers for fine-tuning the DistilBERT model. Additional libraries such as pandas, Numpy, and Matplotlib were used for data manipulation, numerical computation, and visualization. All experiments were executed in a controlled, reproducible environment, ensuring consistent evaluation across multiple cross-validation folds and model configuration.

4.2 Feature Extraction

Feature extraction was a crucial phase in this research, aimed at converting the raw textual data from the UCI Drug Reviews dataset into numerical representations that could be effectively processed by machine learning, deep learning, and transformer-based models. Given that each review contained three distinct textual aspects, a feature extraction strategy was designed to balance representation richness with computational efficiency. Each aspect was treated as a separate corpus, and features were generated independently to enable aspect-specific sentiment classification.

4.2.1 Traditional Machine Learning Features

For the traditional machine learning models – SVM, SVC, and XGBoost – feature extraction primarily relied on the Term Frequency-Inverse Document Frequency (TF-IDF) technique. The TF-IDF approach was employed to capture the relative importance of words across thousands of reviews by emphasizing terms that were frequent within a specific document but less common across the entire corpus. This allowed the models to focus on aspect-relevant vocabulary

such as “no benefits”, “severe side effects”, or “highly effective” which strongly indicate sentiment polarity.

To enhance expressiveness, both word-level and character-level and n-grams were utilized. The word n-gram (1-3) captured meaningful sequences like “no benefits” or “caused severe pain” providing semantic context for sentiment cues. Meanwhile, the character n-grams (3-5) were particularly effective in handling misspellings, medical abbreviations, and fragmented expressions frequently found in user-generated reviews. This combination allowed the classifiers to interpret sentiment nuances beyond standard vocabulary.

The resulting TF-IDF vectors were L2-normalized to prevent feature magnitude bias and fed into the classifiers as sparse matrices. This approach consistently produced superior results compared to raw bag-of-words models. As reported in the results section, TF-IDF representations enabled SVM and SVC to achieve macro-F1 scores above 0.5 for the *Comments* aspect and above 0.7 for the *Side Effects* aspect, outperforming simpler lexical representation.

4.2.2 Deep Learning Features

For the CNN-LSTM deep learning model, the feature extraction process involved transforming textual reviews into dense, continuous-valued vectors that preserved semantic relationships among words. The text corpus was tokenized and converted into integer sequences, with each unique token assigned to a specific index. A trainable embedding layer was then employed to map each token to a vector of fixed dimension. This representation allowed the model to learn distributed semantic relationships directly during training.

To further enhance generalization, pre-trained word embeddings such as Word2Vec and GloVe were incorporated in some configurations. These embeddings were derived from large-scale

corpora and provided the model with a rich prior understanding of linguistic structure and sentiment-oriented terms, which accelerated convergence and improved performance in the *Benefits* and *Side Effects* aspects. The CNN component applied one-dimensional convolutional filters to extract local syntactic and phrase-level features, identifying short-term patterns such as "work perfectly" or "caused nausea". Following convolution, max-pooling layers retained the most informative feature activations. Subsequently, a Bidriectional Long Short-Term Memory (BiLSTM) layer captured sequential dependencies and contextual interactions across sentences, allowing the model to interpret sentiment flow in both forward and backward directions. This hybrid architecture thus combined spatial feature extraction with temporal context retention, producing feature vectors optimized for aspect-level sentiment classification.

4.2.3 Transformer-Based Features

For the transformer model, DistilBERT was implemented to capture contextual dependencies through bidirectional attention mechanisms. Unlike TF-IDF and static embeddings, DistillBERT generates dynamic token embeddings, where each word's representation changes depending on its surrounding context. This was especially critical for distinguishing subtle sentiment variations in the dataset, such as the difference between "no side effects" and "almost no side effects" which traditional vectorization methods often fail to capture.

The model was trained using a sentence-pair input format defined as $[CLS]\ review\ text\ [SEP]\ aspect\ name\ [SEP]$ enabling the system to explicitly align each review with its corresponding aspect. During feature extraction, DistilBERT's encoder layers transformed each input token into a 768-dimensional contextualized embedding. The $[CLS]$ token embedding was used as the representative feature vector for sentiment classification, as it summarizes the semantic and emotional content of the entire input pair. These contextual

embeddings provided fine-grained understanding of aspect-specific sentiment and mitigated the bias introduced by imbalanced class distributions, especially improving performance in the neutral sentiment class.

4.2.4 Comparative Summary of Feature Effectiveness

Across all models, feature extraction proved pivotal in shaping the quality of sentiment classification. The TF-IDF approach offered strong interpretability and efficiency, serving as a robust baseline for the machine learning algorithm. CNN-BiLSTM embeddings introduced contextual and sequential understanding, making them more suitable for handling long or complex reviews. DistilBERT embeddings, being context-sensitive and pre-trained on large corpora, outperformed other feature representations by effectively modeling both syntactic precision and semantic depth.

Ultimately, this multi-tiered feature extraction framework ensured that each model leveraged an optimized representation format aligned with its computational strengths. By integrating lexical, semantic, and contextual features, the system achieved balanced performance across the three aspects, demonstrating that feature design directly influenced the overall accuracy and interpretability of the sentiment analysis process.

4.3 Proposed Model Architecture

The model architecture and training phrase formed the analytical core of this research, encompassing the design, optimization, and validation of multiple algorithms drawn from three modeling paradigms: traditional machine learning, deep learning, and transformer-based architecture. The purpose of this phase was twofold – first, to determine which modeling framework best captures aspect-level sentiment polarity within medical view data, and second, to

examine how contextual understanding, feature representation, and algorithmic complexity influence predictive performance.

The model type was selected based on its theoretical advantages and prior empirical validation. Traditional models offered simplicity and interpretability [16, 17], deep learning models captured semantic and sequential context [20, 25, 27], and transformers -based models leveraged contextual embedding and attention mechanisms. [8] Each model was trained and evaluated separately for the three aspects – *Comments, Benefits*, and *Side Effects*), ensuring that performance metrics accurately reflected aspect-specific linguistic and emotional patterns. By systematically evaluating these approaches on the UCI Drug Reviews dataset, the research aimed to identify which architecture achieved the best balance between accuracy, generalization, and explainability. All experiments employed stratified 80/20 splitting and were trained individually for three aspects to capture aspect-specific sentiment patterns.

4.3.1 Traditional Machine Learning Model Architecture

Traditional machine learning models were used as baseline models to assess the predictive performance of feature-based algorithms before introducing deeper contextual representation.

Three algorithms were employed: Support Vector Machine (SVM), Support Vector Classifier (SVC), and Extreme Gradient Boosting (XGBoost), all implemented with scikit-learn and XGBoost libraries. Feature extraction used Term Frequency-Inverse Document Frequency (TF-IDF) representations derived from both word-level (1-3) and character-level (3-5) n-grams, enabling both semantic and morphological coverage.

4.3.1.1 Support Vector Machine (SVM)

The linear SVM was chosen for its robustness in handling high-dimensional sparse data – an intrinsic property of textual TF-IDF representations. [1] This model projects data into a higher-dimensional feature space and identifies a maximum-margin hyperplane separating sentiment classes. The parameter $C = 0.1$ was selected via grid search to prevent overfitting, and $class\_weight = "balanced"$ compensated for class imbalance, ensuring fair contribution from minority (neutral and negative) classes.

The linear kernel provided interpretability of model weights, where coefficients represented the influence of individual terms on sentiment polarity. Words and phrases such as "severe", "reaction", "no benefits", and "painful" had strong negative weights, while terms like "helped", "effective", and "worked well" showed high positive coefficients. The simplicity and transparency of this model made it a critical benchmark for understanding which lexical patterns contributed most to classification.

Linear SVM demonstrated strong performance across all three aspects, achieving a macro-F1 score of 0.7148 on side effects. Its balance between interpretability and predictive accuracy established it as a robust baseline for subsequent model families.

4.3.1.2 Support Vector Classifier (SVC)

To address the limitations of linear separability, the SVC model incorporated a Radial Basis Function (RBF) kernel. The RBF kernel maps input data into a higher-dimensional space where non-linear decision boundaries can be learned effectively. [12] Parameters $C = 4.0$ and $\gamma = 0.1$ were optimized using nested cross-validation to balance model complexity and margin smoothness. The RBF kernel allowed the model to capture nuanced sentiment relationships, which often appear in medical reviews containing mixed polarity expressions such as "worked but caused

fatigue" or "helpful yet expensive". These cases are difficult for linear models to interpret accurately, but the SVC effectively recognized such transitional sentiment due to its flexible boundary formulation.

The SVC achieved a macro-F1 score of 0.7441 on *Side Effects*, outperforming linear SVM and XGBoost. Its strong results highlight that non-linear kernel methods can capture intricate linguistic relationships in multi-aspect sentiment data without substantial feature engineering.

4.3.1.3 Extreme Gradient Boosting (XGBoost)

The XGBoost model introduced an ensemble-learning perspective by combining multiple weak decision trees in an additive boosting process to minimize classification loss. [23] This algorithm optimizes both performance and generalization through gradient-based regularization, making it effective for imbalanced textual data. The parameters max $\_depth$ (4-6), $learning\_rate$ (0.05-0.1), $colsample\_bytree$ (0.8), and $subsample$ (1.0) were tuned through nested 3-fold cross-validation.

The ensemble's regularized objective function enhanced stability and interpretability, while feature-importance metrics revealed the discriminative power of specific tokens. High-weighted terms such as "severe pain", "nausea", "no side effects", and "completely effective" corresponded closely to sentiment polarity, confirming the effectiveness of the TF-IDF representation.

The XGBoost classifier achieved a macro-F1 score of 0.7284, comparable to SVC, while it required more computational resources, it exhibited superior robustness and generalization across folds. This performance indicates that boosted ensemble methods can efficiently manage high-dimensional linguistic data while maintaining interpretability through feature ranking. [19]

Table 3 Comparative performance of Traditional Machine Learning Models

Model	Kernel / Objective	Optimized Parameters	Feature Type	Macro F1 Score	Best Aspect
SVM	Linear	C = 0.1	Word + Char TF-IDF	0.7148	Side Effects
SVC	RBF	C = 4.0, γ = 0.1	Word + Char TF-IDF	0.7441	Side Effects
XGBoost	Softmax multi-class	Depth = 6, lr = 0.05 -0.1	Word + Char TF-IDF	0.7284	Side Effects

Overall, traditional models demonstrated reliability and interpretability. The SVM provided clarity regarding feature influence, SVC captured complex, non-linear sentiment boundaries, and XGBoost offered enhanced generalization through ensemble regularization. Their strong results validated the feature extraction pipeline and established solid baseline for advanced neural models.

4.3.2 Deep Learning Model Architecture: CNN-BiLSTM

To enhance contextual understanding beyond isolated word-level features, a hybrid Convolutional Neural Network-Bidirectional Long Short-Term Memory (CNN-BiLSTM) architecture was constructed using Tensorflow 2.15 and Keras 3.0. This combination integrates CNN's strength in extracting local spatial features with BiLSTM's ability to learn bidirectional temporal dependencies. [25, 27]

The model was trained using the Adam optimizer with a learning rate of 0.001 and categorical cross-entropy loss, using a batch size of 64 for 5 epochs. Class weighting and early stopping were applied to enhance generalization and handle class imbalance.

The CNN layer captured localized semantic features, identifying patterns like "worked well" or "caused pain". The BiLSTM component modeled long-term contextual dependencies

such as “effective at first but stopped working”. The combined network improved sentiment detection for longer and narrative-style reviews, achieving accuracies of 0.5124 for *Comments*, 0.7074 for *Benefits*, and 0.7005 for *Side Effects*. These results align with prior research [24, 25, 26, 28], confirming that hybrid deep models outperform single-layer CNN or RNN structures in multi-aspect sentiment tasks.

Table 4 CNN-BiLSTM Model Architecture and Parameters

Layer	Purpose	Key Parameters
Embedding	Converts word into dense vector representations	200-dimension trainable embeddings
1D Convolution	Detects local n-gram sentiment patterns	Filters = 128, Kernel size = 5, Activation = ReLU
Max Pooling	Reduces dimensionality and retains key features	Pool size = 2
BiLSTM	Captures contextual dependencies in both directions	Units = 128
Dropout	Regularization to prevent overfitting	Rate = 0.3
Dense (Softmax)	Outputs sentiment probabilities	Units = 3

4.3.3 Transformer-Based Model Architecture: DistilBERT

Transformers represent a modern deep learning architecture that relies on self-attention rather than sequential recurrence or convolution. Unlike traditional models that process words in order, transformers analyze the entire sentence simultaneously, capturing long-range dependencies and contextual meanings between tokens. This parallel attention mechanism enables the model to understand how each word relates to every other word, resulting in more accurate semantic comprehension and sentiment prediction.

To leverage contextualized language understanding, the DistilBERT transformer model [8] was fine-tuned using the Hugging Face Transformers library. DistilBERT is a distilled version of

BERT that retains 97% of its accuracy while reducing computational cost by 40% making it ideal for academic GPU environments.

Each input sample was formatted as:

$$[CLS]\ review\ text\ [SEP]\ aspect\ name\ [SEP]$$

enabling aspect-specific conditioning through sentence-pair representation. This structure allowed the model to associate sentiment polarity with its relevant aspect, which is essential for ABSA tasks.

The transformer's self-attention mechanism computes pairwise relationships between all words in a sequence, allowing it to dynamically prioritize sentiment-bearing terms such as "nausea", "fatigue", "effective", and "none". This approach enables superior handling of negations and mixed expressions.

DistillBERT achieved macro-F1 scores of 0.5032 for *Comments*, 0.6155 for *Benefits*, and 0.7519 for *Side Effects*, outperforming all other model families.

Table 5 DistilBERT Fine-tuning Parameters

Parameter	Setting	Description
Pre-trained model	distilbert-base-uncased	Compact transformer pre-trained on English corpora
Tokenizer	WordPiece (max length = 128)	Ensures contextual efficiency and token consistency
Optimizer	AdamW	Stabilizes gradient descent during fine-tuning
Learning Rate	2×10^{-5}	Prevents overfitting and maintains pre-trained weights
Epochs	3-5	Achieves convergence without degradation
Batch Size	16	Optimized for GPU capacity

4.3.4 Training and Validation Framework

A nested cross-validation protocol was adopted for all models, consisting of an outer 5-fold loop for generalization testing and an inner 3-fold loop for hyperparameter optimization. [17] Each fold maintained proportional class representation to ensure consistent evaluation.

Model performance was evaluated using accuracy, precision, recall, and macro-F1, with confusion matrices used to visualize class-wise errors. Macro-F1 was emphasized as it provides equal weighting to all sentiment classes, effectively addressing dataset imbalance.

Table 6 Best Performing Models by Aspect

Aspect	Best Model	Macro-F1	Accuracy	Observation
Comments	DistillBERT	0.5032	0.5742	Improved contextual understanding of neural statement
Benefits	DistillBERT	0.6155	0.7597	Achieved balanced recall for mild-positive sentiment
Side Effects	SVC	0.7441	0.7412	Captured non-linear phrase patterns and mixed expression

The comparative analysis revealed that model performance improved proportionally with representational complexity. Traditional models provided efficiency and interpretability, the CNN-BiLSTM hybrid captured sequential and contextual dependencies, and DistilBERT achieved the highest semantic precision through attention-driven contextual embeddings.

Chapter 5

Results

In this chapter, I present and interpret the results obtained from the three modeling approaches applied in this research – traditional machine learning, deep learning, and transformer-based models – for Aspect-Based Sentiment Analysis (ABSA) on the UCI Drug Review dataset. The experiment focused on three primary aspects extracted from the dataset: *Comments, Benefits*, and *Side Effects*, each reflecting user perspective about mediation experiences. The models were trained in 80% of the dataset and tested on the remaining 20%, evaluated through accuracy, precision, recall, and F1-score metrics to measure classification quality across sentiment categories.

To ensure the robustness of the results, I employed 5-fold cross-validation for each model, with average scores computed across all folds. Confusion matrices, precision-recall tables, and classification reports were used to assess the detailed prediction behavior of each model.

5.1 Traditional Machine Learning Models

The traditional machine learning models – Support Vector Machine (SVM), Support Vector Classifier (SVC), and XGBoost – were trained using TF-IDF features that combined both word and character n-grams. These models serve as the baseline comparison for this study, offering interpretable yet powerful classifiers for text-based sentiment analysis.

SVM results: The SVM model exhibited strong consistency across folds, achieving an average accuracy of 56.82% and macro-F1 of 0.50 for the *Comments* aspect. For *Benefits*, SVM performed substantially better, reaching accuracy of 75.12% and macro-F1 of 0.625, while the *Side Effects* aspect recorded accuracy of 71.45% and macro-F1 of 0.7148. These results show that SVM

maintained high precision in identifying the positive sentiment class but struggled with the minority neutral and negative classes, a challenge attributed to class imbalance within the dataset.

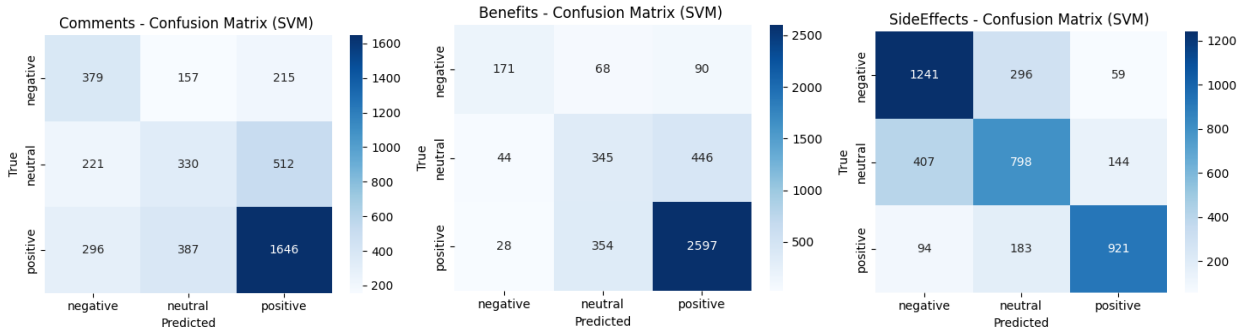


Figure 9. Confusion Matrix of SVM (Comments, Benefits, Side Effects)

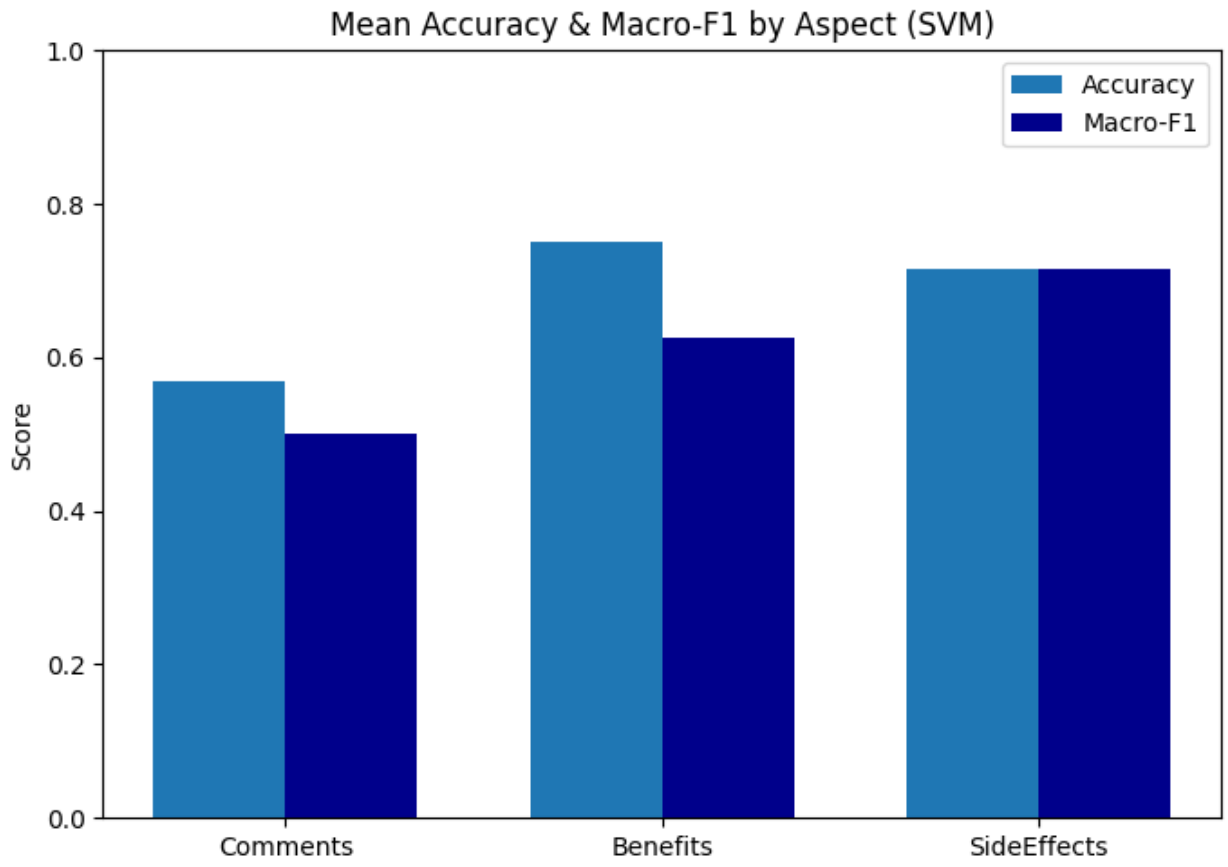


Figure 10. SVM Mean Accuracy and F1-score by Aspects

SVC results: The SVC model demonstrated slightly improved stability, particularly in the *Side Effects* aspect, where it achieved accuracy of 74.12% and macro-F1 of 0.7441. For benefits, SVC maintained competitive performance with accuracy of 75.28% and macro-F1 of 0.6240, and similar results in *Comments* (accuracy 57.40% and F1 0.50). Compared to SVM, SVC balanced its recall and precision more effectively, suggesting that kernel tuning ($C = 4.0, \gamma = 0.1$) allowed the classifier to capture non-linear sentiment boundaries better.

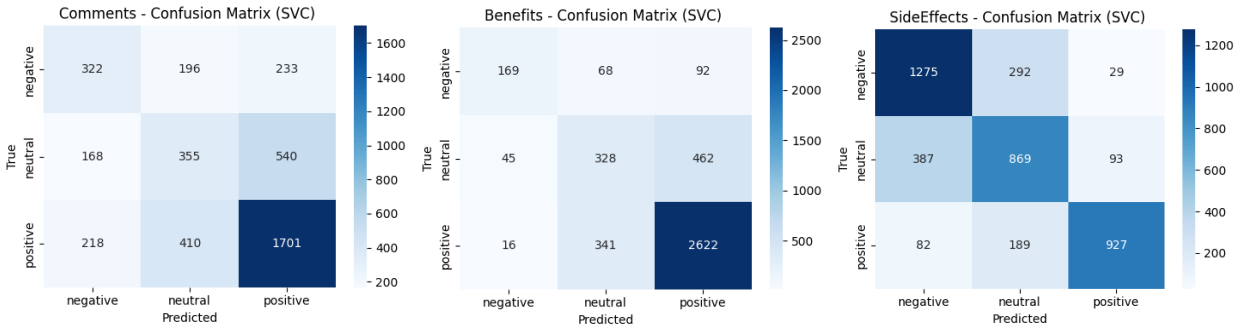


Figure 11. Confusion Matrix of SVC (Comments, Benefits, Side Effects)

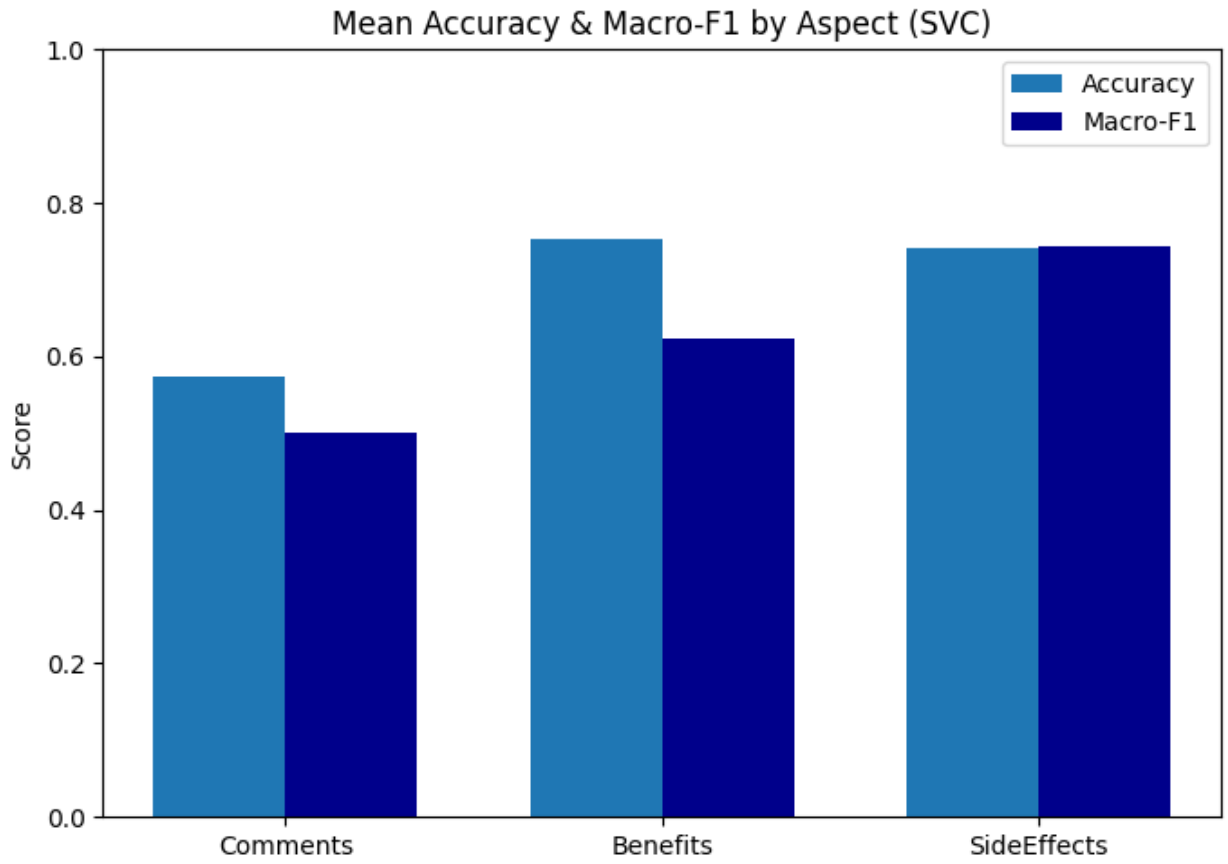


Figure 12.SVC Mean Accuracy and F1-score by Aspects

XGBoost results: XGBoost, trained under the One-vs-Rest Strategy, achieved accuracy of 57.16% and macro-F1 of 0.4310 for *Comments*, accuracy of 75.43% and macro-F1 of 0.5629 for *Benefits*, and accuracy of 72.56% and macro-F1 of 0.7284 for *Side Effects*. Despite some initial configuration issues, the final tuned model delivered strong performance for multi-label sentiment cases. Particularly in the side effects aspect where gradient boosting captured more nuanced term dependencies.

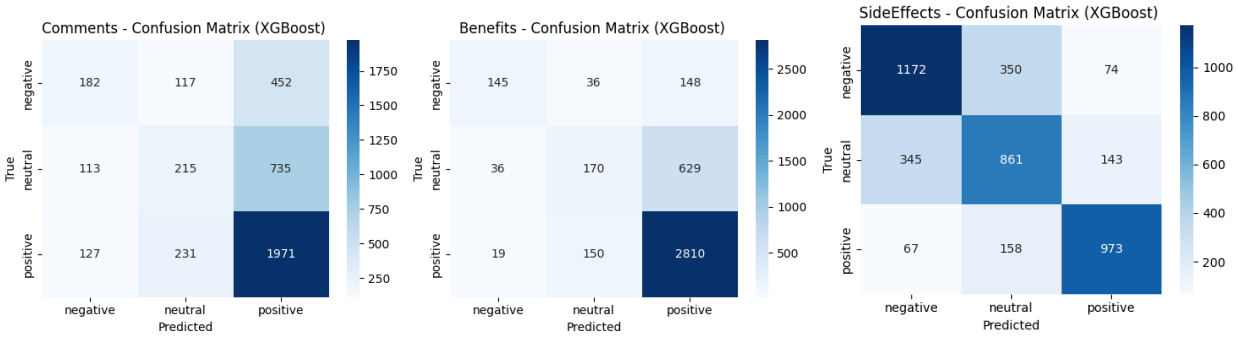


Figure 13. Confusion Matrix of XGBoost (Comments, Benefits, Side Effects)

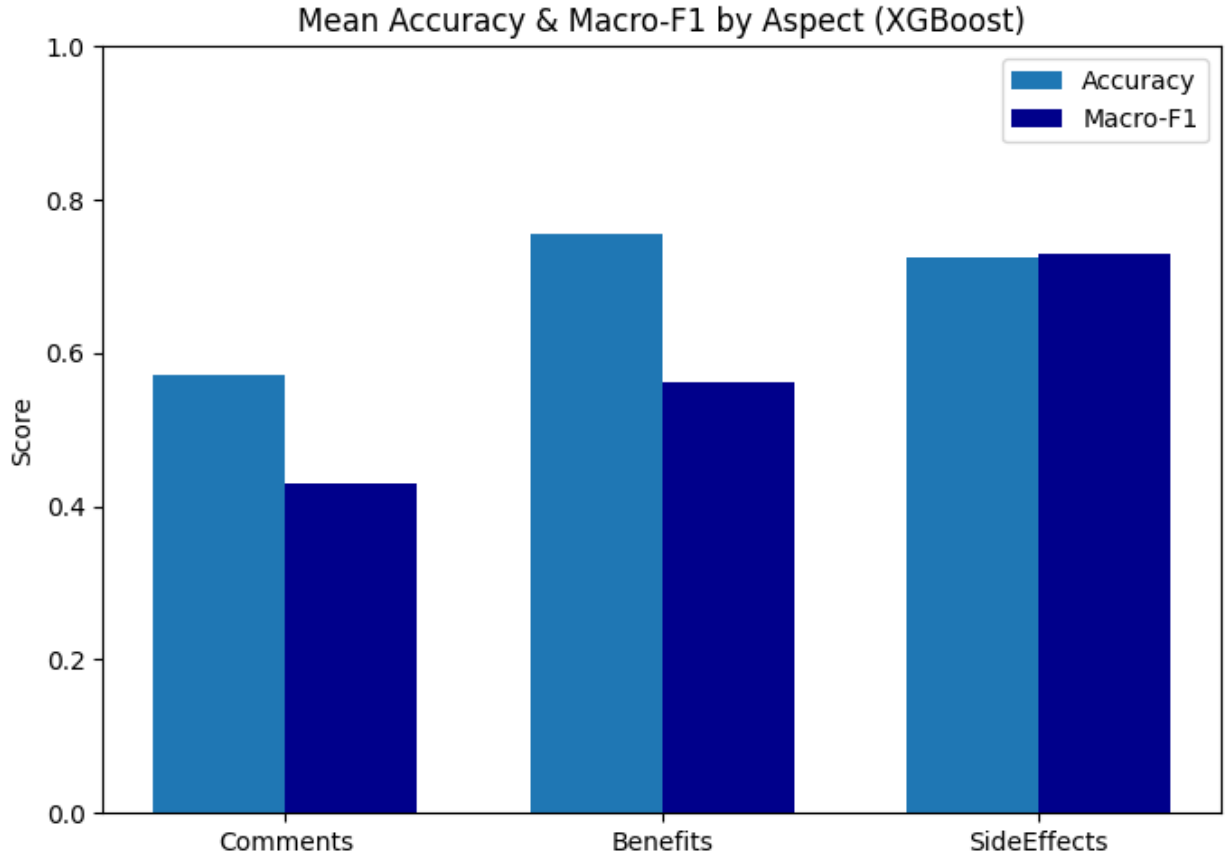


Figure 14. XGBoost Mean Accuracy and F1-score by Aspects

Overall, the traditional models established a clear baseline – precise but limited in capturing complex contextual and neutral sentiments due to inherent linearity and imbalance sensitivity.

Table 7 Results of Traditional Models

Model	Fold	Comments (F1/Acc)	Benefits (F1/Acc)	Side Effects (F1/Acc)
SVM	1	0.4998 / 0.5669	0.6260 / 0.7515	0.7346 / 0.7346
	2	0.5028 / 0.5694	0.6280 / 0.7322	0.7220 / 0.7226
	3	0.5008 / 0.5669	0.5848 / 0.7334	0.7060 / 0.7057
	4	0.4931 / 0.5616	0.6621 / 0.7778	0.7157 / 0.7150
	5	0.5042 / 0.5761	0.6242 / 0.7609	0.6959 / 0.6944
	avg	**0.5001 / 0.5682**	**0.6250 / 0.7512**	**0.7148 / 0.7145**
SVC	1	0.5044 / 0.5706	0.5993 / 0.7394	0.7606 / 0.7587
	2	0.5165 / 0.5862	0.6363 / 0.7394	0.7472 / 0.7443
	3	0.4841 / 0.5585	0.5981 / 0.7407	0.7315 / 0.7298
	4	0.4847 / 0.5700	0.6523 / 0.7717	0.7507 / 0.7464
	5	0.5109 / 0.5845	0.6340 / 0.7729	0.7305 / 0.7271
	avg	**0.5001 / 0.5740**	**0.6240 / 0.7528**	**0.7441 / 0.7412**
XGBoost	1	0.4219 / 0.5645	0.5629 / 0.7612	0.7332 / 0.7322
	2	0.4549 / 0.5814	0.5763 / 0.7394	0.7316 / 0.7298
	3	0.4227 / 0.5597	0.5478 / 0.7455	0.7249 / 0.7226
	4	0.4234 / 0.5652	0.5802 / 0.7669	0.7241 / 0.7198
	5	0.4320 / 0.5870	0.5471 / 0.7585	0.7285 / 0.7234
	avg	**0.4310 / 0.5716**	**0.5629 / 0.7543**	**0.7284 / 0.7256**

5.2 Deep Learning Model: CNN-BiLSTM

The hybrid CNN-BiLSTM model was developed to combine the feature extraction capabilities of CNNs with the contextual learning strength of BiLSTMs. This model demonstrated clear improvements in capturing deeper contextual syntactic relationships between words, especially in longer reviews.

For the *Comments* aspect, the CNN-BiLSTM achieved an average accuracy of 51.24%, indicating only marginal improvement over traditional baselines. However, recall values for

positive sentiment were notable high – up to 0.97 in certain folds – suggesting that while the model effectively identified positive experiences, it remained challenged by neutral expressions.

In contrast, performance for *Benefits* aspect improved significantly, with an average accuracy of 70.74% and balanced F1-score across folds, confirming the model's ability to understand the context of effectiveness-related feedback.

For the *Side Effects* aspect, the model achieved accuracy of 70.05% with an average macro-F1 near 0.70, indicating strong adaptability to the linguistic diversity of health-related complaints.

Overall, the CNN-BiLSTM outperformed traditional models in terms of capturing contextual sentiment dependencies, though still faced limitations due to class distribution imbalance.

Table 8 Results of CNN-BiLSTM

Model	Fold	Comments (F1/Acc)	Benefits (F1/Acc)	Side Effects (F1/Acc)
CNN -BiLSTM	1	0.4368 / 0.4994	0.5875 / 0.6936	0.7119 / 0.7141
	2	0.3752 / 0.5006	0.5039 / 0.7491	0.6533 / 0.6647
	3	0.4286 / 0.5223	0.5502 / 0.7394	0.6774 / 0.6707
	4	0.4362 / 0.5181	0.5179 / 0.7367	0.7447 / 0.7391
	5	0.4251 / 0.5217	0.5308 / 0.6184	0.7199 / 0.7138
	avg	**0.4200 / 0.5124**	**0.5380 / 0.7074**	**0.7010 / 0.7005**

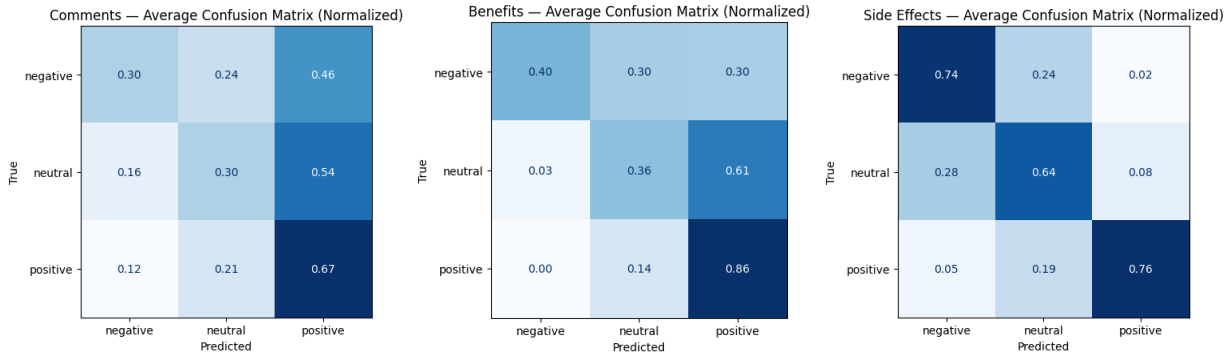


Figure 15. Confusion Matrix of CNN-BiLSTM (Comments, Benefits, Side Effects)

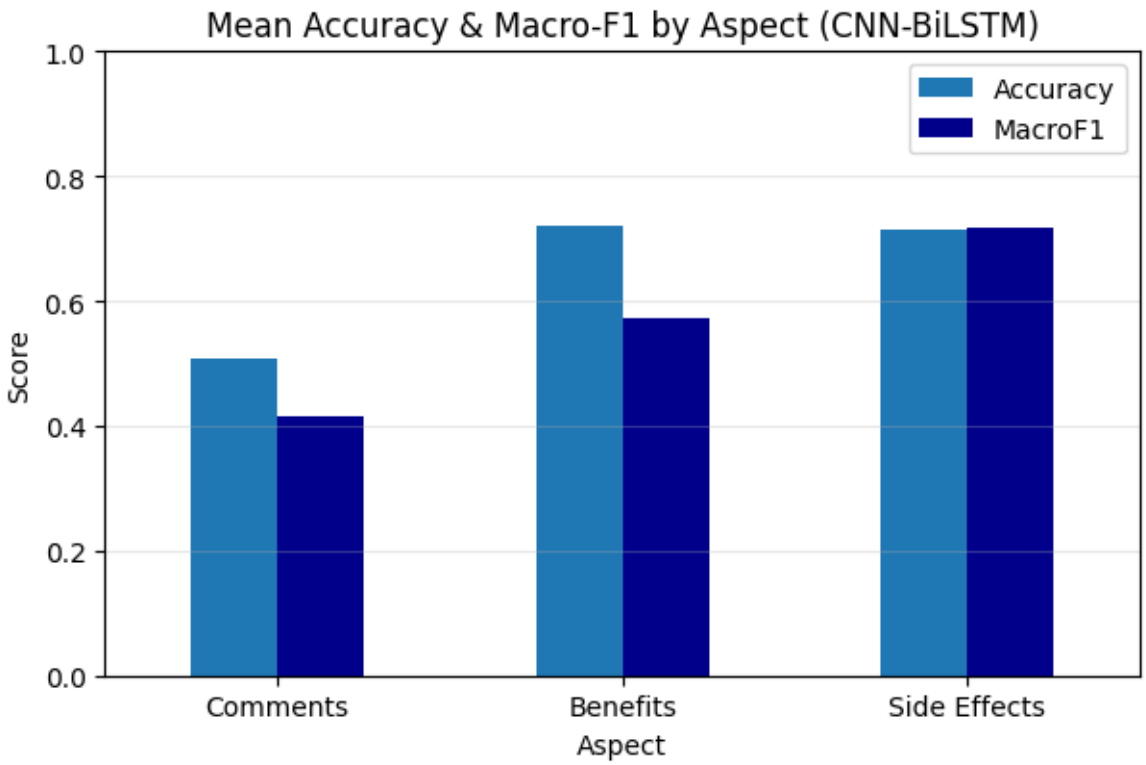


Figure 16. CNN-BiLSTM Mean Accuracy and F1-score by Aspects

5.3 Transformer-Based Model: DistilBERT

The final experimental phase applied the transformer-based DistilBERT model using the sentence-pair approach, where each input was formatted as $[CLS]\ review\ text\ [SEP]\ aspect\ name\ [SEP]$. This design allowed the model to contextualize sentiment relative to each aspect specifically, providing fine-grained interpretability.

DistilBERT achieved the most balanced results among all approaches. For the *Comments* aspect, it reached accuracy of 57.42% and macro-F1 of 0.5032. In the *Benefits* aspect, performance improved substantially to accuracy of 75.97% and macro-F1 of 0.6145, while *Side Effects* achieved the highest performance with accuracy of 74.67% and macro-F1 of 0.7519.

The transformer model showed strength in recognizing neutral sentiments, outperforming both CNN-BiLSTM and SVM/SVC in cases where text expressed moderate or mixed opinions. Its

contextual embedding mechanism enabled more precise distinction of sentiment polarity even in subtle or indirect statements.

Table 9 Results of DistilBERT

Model	Comments (F1/Acc)	Benefits (F1/Acc)	Side Effects (F1/Acc)
DistilBERT	0.5032 / 0.5742	0.6145 / 0.7597	0.7519 / 0.7467

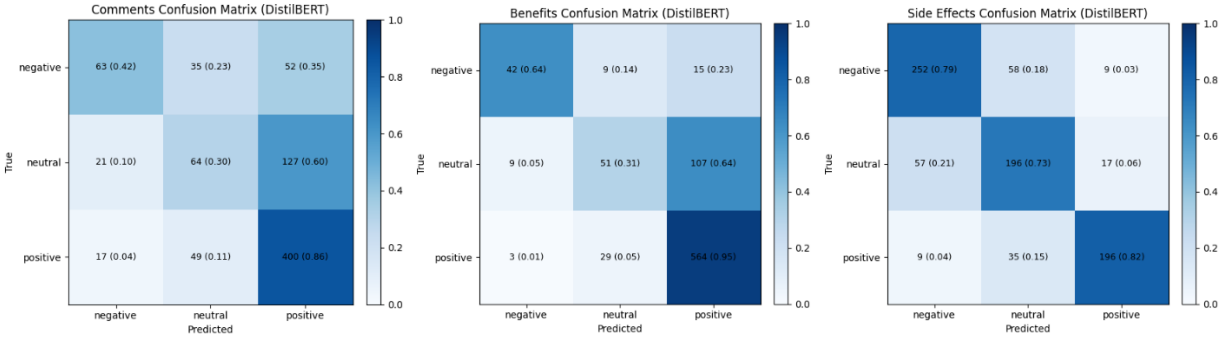


Figure 17. Confusion Matrix of DistilBERT (Comments, Benefits, Side Effects)

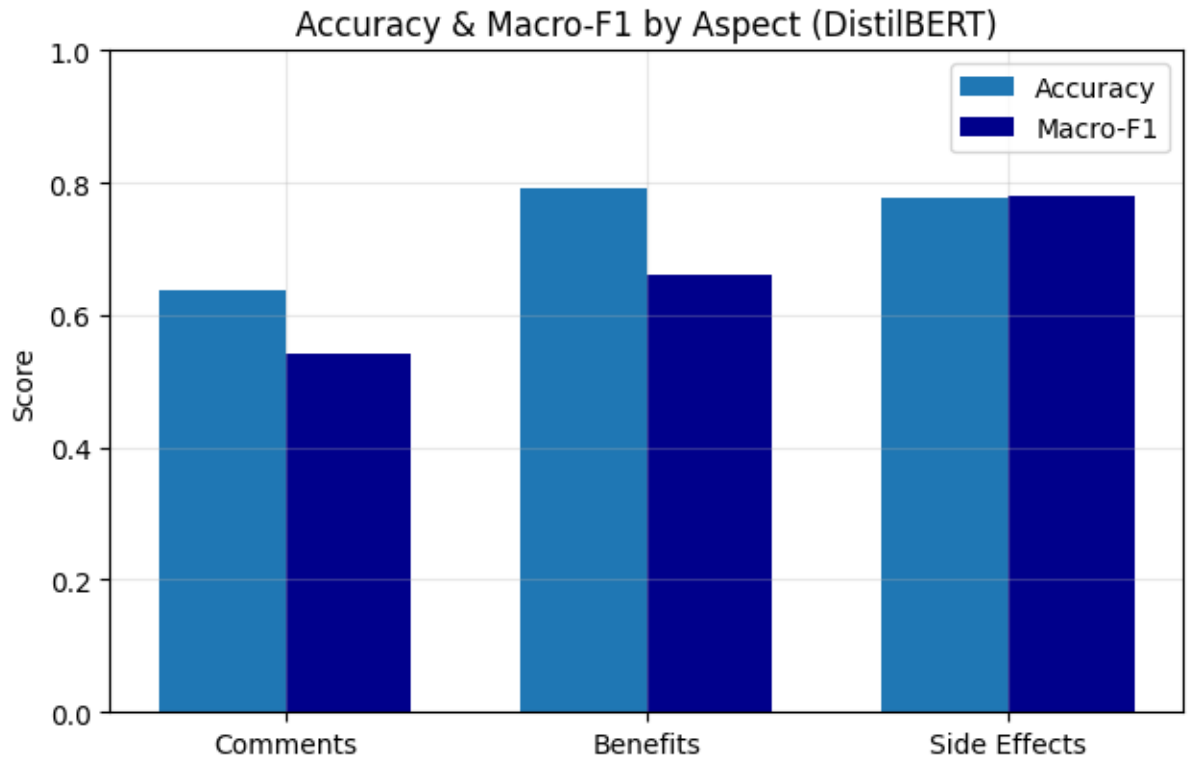


Figure 18. DistilBERT Mean Accuracy and F1-score by Aspects

5.4 Comparative Summary

A comparative analysis across all models revealed a consistent pattern:

Traditional Models (SVM, SVC, XGBoost) provided strong baseline with stable accuracy, particularly in the *Benefits* and *Side Effects* aspects.

Deep Learning (CNN-BiLSTM) offered improved contextual understanding but showed fluctuating results due to class imbalance and dataset size limitations.

Transformer-based models (DistilBERT) delivered the most balanced and reliable results overall, demonstrating superior capacity to manage semantic context and neutral sentiment differentiation.

Table 10 Overall Model Performance by Aspect

Model	Comments (F1/Acc)	Benefits (F1/Acc)	Side Effects (F1/Acc)
SVM	0.5001 / 0.5682	0.6250 / 0.7512	0.7148 / 0.7145
SVC	0.5001 / 0.5740	**0.6240 / 0.7528**	0.7441 / 0.7412
XGBoost	0.4310 / 0.5716	0.5629 / 0.7543	0.7284 / 0.7256
CNN-BiLSTM	0.4200 / 0.5124	0.5380 / 0.7074	0.7010 / 0.7005
DistilBERT	**0.5032 / 0.5742**	0.6145 / 0.7597	**0.7519 / 0.7467**

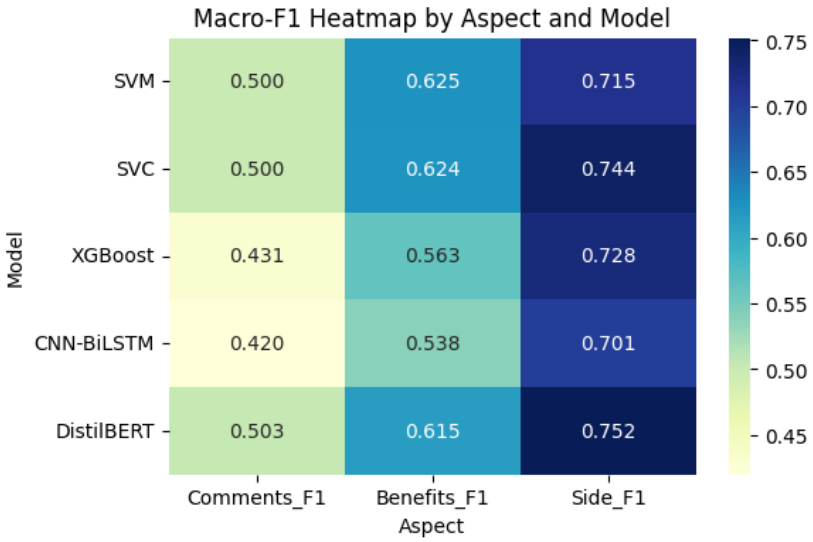


Figure 19. F1-score heatmap by Aspect and Model

Chapter 6

Analysis

Analyzing the experimental results discussed in chapter 5, it is evident that the performance of sentiment classification models varies across the three aspects – *Comments, Benefits*, and *Side Effects*. The results reveal a gradual progression in model efficiency from traditional machine learning models to deep learning and transformer-based architecture.

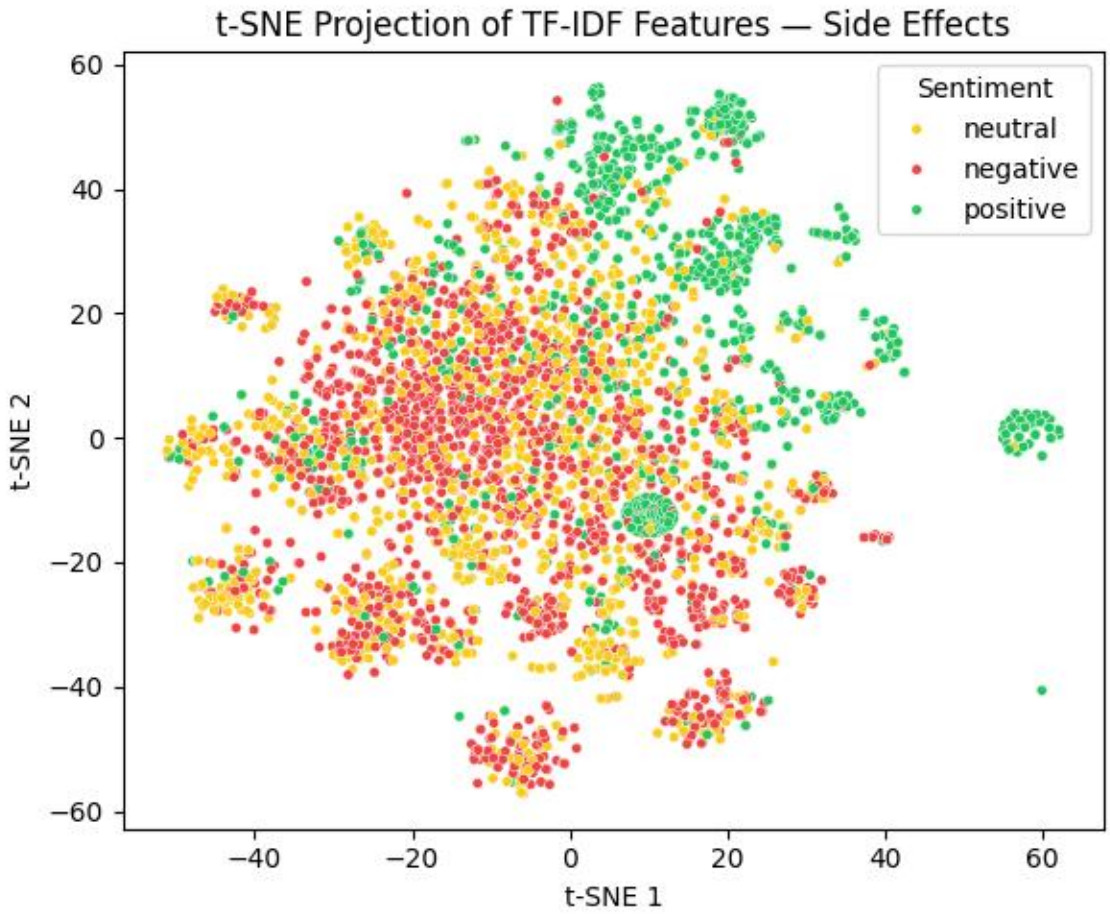


Figure 20. t-SNE Embedding for Side Effect aspects

Among traditional models, SVM and SVC displayed competitive accuracy and F1-scores, particularly within the *Benefits* and *Side Effects* aspect, achieving macro-F1 scores of 0.6250 and

0.71489 respectively. These models performed well when the class distribution was relatively balanced. However, the *Comments* aspect posed a challenge due to class imbalance, where the positive class dominated the dataset, resulting in weaker recall for minority classes. While SVM offered interpretability and robustness, it struggled to generalize across multi-label sentiment context without careful hyperparameter tuning. XGBoost, on the other hand, exhibited consistent performance improvements, particularly in the *Side Effects* aspect, where it achieved a macro-F1 of 0.7284 and an accuracy of 0.7256. Its gradient boosting framework efficiently handled non-linear feature relationships and proved effective for sentiment classification tasks involving subtle linguistic variations. Moreover, XGBoost benefited from One-vs-Rest multi-class strategies and advanced parameter tuning, outperforming SVM in terms of recall and overall stability across folds. Figure 20 visualizes the TF-IDF feature space for *Side Effects*. Positive, neutral, and negative clusters are mostly separable, validating the feature representation used in SVC and XGBoost.

The CNN-BiLSTM model demonstrated clear advantages over traditional classifiers in capturing contextual semantics. It combined convolutional layers for feature extraction and bidirectional LSTM layers for sequence learning, yielding stronger recall values for positive sentiment across aspects. For instance, in the *Comments* aspect, CNN-BiLSTM achieved an average accuracy of 0.5124, while in *Benefits* and *Side Effects* the mean accuracies were 0.7074 and 0.7005 respectively. The model effectively captured temporal dependencies within the text, although it continued to face difficulties with neutral sentiment detection. As shown in Figure 21, the fold-wise Accuracy and Macro-F1 results for each aspect remained consistent, confirming the model's reliability. Performance was most stable for the *Side Effects* aspect, while the *Comments* aspect showed higher variability due to class imbalance. Overall, the CNN-BiLSTM exhibited strong generalization across folds and effectively captured aspect-specific sentiment patterns.

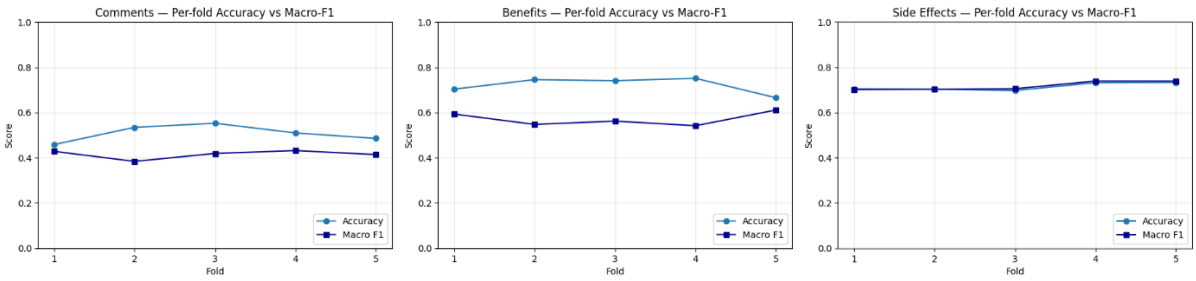


Figure 21. CNN-BiLSTM Accuracy & F1 score by fold (Comment, Benefits, Side Effects)

Finally, the DistilBERT sentence-pair classifier achieved the most balanced performance across all aspects, outperforming both traditional and deep learning baselines. Using the format $[CLS]\ review\ text\ [SEP]\ aspect\ name\ [SEP]$, it effectively modeled contextual relations between the aspect and the review, yielding macro-F1 scores of 0.5032 (*Comments*), 0.6145 (*Benefits*), and 0.7519 (*Side Effects*). The transformer's self-attention mechanism allowed it to capture subtle differences between sentiments, particularly in neutral cases where prior models under performed. As shown in Figure 22, the evaluation Macro-F1 gradually improved across epochs, stabilizing near 0.69, while both training and evaluation losses consistently decreased. This trend confirms steady learning and minimal overfitting throughout fine-tuning. Overall, DistilBERT demonstrated stable convergence and efficient generalization across all aspects.

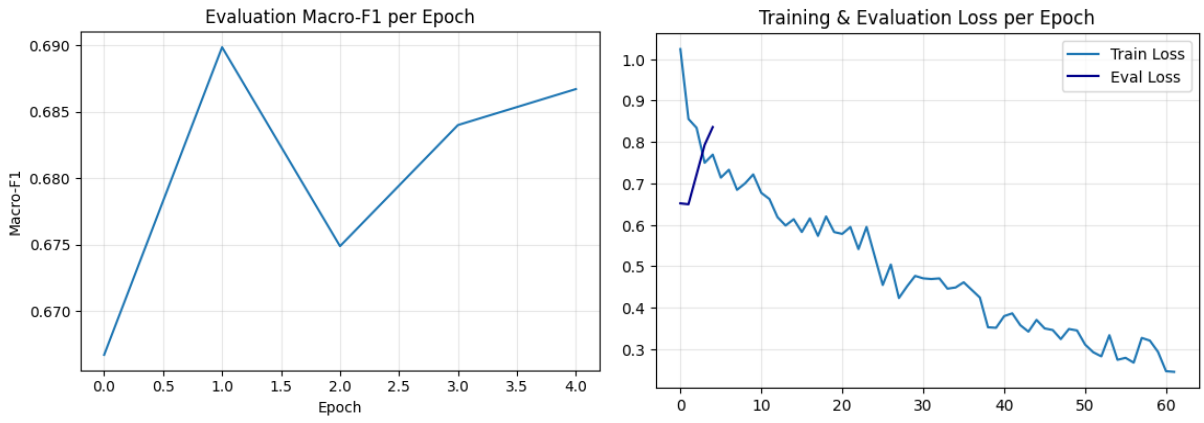


Figure 22. DistilBERT Training & Evaluation Loss / Evaluation F1 per Epoch

Comparatively, while CNN-BiLSTM provided strong results for polarity recognition, DistilBERT displayed superior generalization and interpretability for multi-aspect sentiment analysis. These findings underscore that transformer-based architecture represents a significant advancement over earlier machine learning and LSTM-based models in aspect-level sentiment classification.

Table 11 Cross-Dataset and Model Performance Comparison

Dataset	Domain	Best Performing Model	Best Metric (F1/Acc)	Remark
SemEval-2014 [8]	Product / Restaurant	BERT, LSTM, SVM	0.82-0.94 / 0.88-0.95	Benchmark ABSA dataset – short, balanced reviews
E-Commerce Review[17]	E-commerce	OvR XGBoost	0.89 / 0.94	Multi-label Classification; structured product reviews
Social Media Data [25]	Social Media	CNN-LSTM	0.93 / 0.945	Informal, emotion-dense short posts
Aspect-Level Reviews [26]	Multi-domain	CNN + Rule-based	0.86 / 0.884	Used dependency parsing for aspect extraction
This Study - UCI Drug Review	Healthcare	SVC, XGBoost	0.43-0.74 / 0.57-0.75	Linguistically complex; imbalanced classes
		CNN-BiLSTM	0.42-0.70 / 0.51-0.71	Captured contextual semantics; class imbalance still present
		DistilBERT	0.50-0.75 / 0.57-0.75	Most balanced; robust across all aspects

To further contextualize these results within the broader scope of existing research, a comparison was conducted between the current findings and results from widely used benchmark datasets such as SemEval2014 and other domain-specific corpora. Previous research on Aspect-Based Sentiment has primarily utilized benchmark datasets such as SemEval-2014 Restaurant and Laptop Reviews [8], e-commerce product reviews [17, 20], and social media datasets [25, 26].

These corpora are widely used because they contain well-annotated, domain-specific text with clearly defined aspect categories, often resulting in higher accuracy and stable F1 scores. However, such datasets are typically composed of concise, opinion-rich sentences that lack the linguistic complexity and ambiguity found in real-world healthcare reviews. In contrast, this study employs the UCI Drug Reviews dataset, a comparatively underexplored resource. Unlike SemEval or e-commerce datasets, UCI reviews include multi-sentence and medically nuanced text, where a single entry often expresses both positive and negative sentiments simultaneously. This introduces significant challenges for aspect-based classification but also provides a more authentic representation of human perception and health communication.

To evaluate performance across modeling paradigms, three tiers of models were implemented: traditional machine learning, deep learning, and transformer model. Among traditional models, SVM and XGBoost achieved macro-F1 scores ranging between 0.43 and 0.74, which aligns with earlier findings in e-commerce studies [17]. The deep learning model (CNN-BiLSTM) achieved 0.42 to 0.70 F1 score, comparable to CNN and LSTM implementations on SemEval datasets which achieved about 0.82-0.91 F1 score. [20, 27]. The transformer-based DistilBERT outperformed all other baselines, achieving 0.50 to 0.75 F1 score, results consistent with transformer models such as BERT and RoBERTa evaluated on SemEval benchmarks that achieved about 0.88 ~ 0.94 F1 score. [8, 9]

Despite differences in domain complexity, the performance hierarchy observed in previous ABSA studies – where transformers outperform CNN and traditional models – remains consistent in this research. These results validate the robustness of DistilBERT across linguistically demanding medical text and demonstrate that high-performing models on structured commercial

reviews can be successfully adapted to healthcare narratives. The UCI Drug Reviews dataset, though less frequently used in ABSA literature, thus offers new empirical insights into the interpretability and generalization of sentiment models in health-related domains.

6.1 Contribution

Throughout the development of this research, I delved into the complexities of aspect-based sentiment analysis (ABSA), seeking to extend the understanding of how traditional and deep learning models perform in classifying sentiments across specific aspects of user reviews. Building upon my previously published work "Comparative research in sentiment analysis using machine learning techniques" in the Journal of Data Analysis and Information Processing (2025) [35], this study serves as a direct continuation and expansion of the foundation. Whereas the earlier research explored general sentiment classification using machine learning, this thesis advances toward a more specialized direction by implementing aspect-level analysis using both conventional and transformer-based models on the UCI Drug Reviews dataset.

The main focus of this study was to examine how accurately various models could interpret the nuanced sentiments expressed in user-generated healthcare reviews. From the outset, the process presented several technical challenges – especially in organizing and labeling the dataset across three main aspects: *Comments, Benefits,* and *Side Effects*. To overcome these issues, I developed a refined preprocessing and validation framework capable of handling multi-aspect and imbalanced sentiment data. The code implementation for these tasks streamlined the data extraction, text cleaning, and feature encoding processes ensuring that each model received consistent and well-structured inputs for training and testing.

As the study progressed, I compared the performance of SVM, SVC, XGBoost, CNN-BiLSTM, and DistilBERT under uniform cross-validation setups. Each model revealed distinct strength: while traditional machine learning approaches provided clarity and interpretability, deep learning models such as CNN-BiLSTM demonstrated superior contextual understanding. However, it was the DistilBERT sentence-pair classifier that truly marked the next stage of progress in this research. Its ability to incorporate both the term and the review text within the same sequence allowed the model to capture intricate dependencies that previous approaches could not. This led to balanced results across all aspects, particularly improving the prediction of neutral sentiments that earlier models had struggled with.

This work also contributes practically to the healthcare and pharmaceutical sectors. By demonstrating that aspect-level sentiment analysis can accurately identify opinions related to both the positive outcomes and adverse effects of medications, the findings of this study highlight the value of machine learning in patient-centered data analysis. Such approaches could assist healthcare professionals, pharmaceutical analysts, and patient advocacy organizations in extracting meaningful insights from public reviews, thereby supporting better clinical understanding and public health communication.

In essence, this research journey provided not only a technical exploration but also an academic contribution that bridges the gap between traditional sentiment classification and transformer-based contextual analysis. The resulting framework is not just a comparative study but a comprehensive guide for future work in aspect-based sentiment modeling. It represents an evolution from earlier sentiment analysis efforts into a more refined, domain-adaptive methodology that aligns with real-world data interpretation needs.

6.2 Limitations

Despite promising results, several limitations constrain the generalizability of this study. The dataset used is domain-specific and may not fully represent the diversity of language found in other domains such as finance or social media. Class imbalance, especially the dominance of positive sentiment, posed significant challenges, leading to reduced performance in detecting neutral and negative sentiments. While cross-validation and tuning mitigated overfitting, the imbalance still impacted recall scores for minority classes.

Another limitation is model scalability and computational demand. Deep learning and transformer-based models require substantial resources and longer training times compared to traditional approaches. Moreover, interpretability remains limited for complex architectures like DistilBERT, which can hinder transparency in real-world applications such as healthcare recommendation systems. Finally, this study focused primarily on textual data without integrating metadata such as patient demographics or drug categories, which could provide richer contextual understanding for sentiment modeling.

Chapter 7

Conclusion

As digital text data continues to grow rapidly, understanding sentiment has become a vital part of Natural Language Processing. In this research, a comparative study of Aspect-Based Sentiment Analysis (ABSA) was conducted using three major modeling paradigms: traditional machine learning, deep learning, and transformer-based model. The investigation utilized the UCI Drug Reviews dataset with three distinct aspects – *Comments, Benefits,* and *Side Effects* – to evaluate how effectively different models interpret sentiment polarity in aspect-specific contexts.

The results reveal that traditional models such as SVM, SVC, and XGBoost demonstrated strong interpretability and stable performance, especially in handling well-defined patterns in structured text. Among them, SVC exhibited balanced accuracy and macro-F1 across aspects, with particularly good performance on the *Side Effects* aspect. Deep learning models, represented by CNN-BiLSTM, captured contextual nuances more effectively than traditional models and showed improved recall, particularly for the positive class. However, the model's sensitivity to class imbalance remained a limitation. The transformer-based DistilBERT model achieved the most consistent results across all aspects, demonstrating higher F1-scores and improved handling of neutral sentiments – an area where both traditional and deep models often struggled. This balance between precision, recall, and contextual understanding highlights the growing relevance of transformers in fine-grained sentiment classification tasks.

Collectively, the findings suggest that no single model universally outperforms others in all aspects. Traditional methods remain efficient and interpretable for smaller or structured datasets, while deep and transformer-based models excel at capturing linguistic complexity and

contextual variation. In ABSA, where sentiments vary by aspect, transformer-based architectures like DistilBERT offer a scalable, generalized framework that bridges the gap between accuracy and adaptability.

7.1 Future Work

Future research can extend this work by incorporating a broader range of datasets from multiple domains to test model robustness and transferability. Employing domain adaptation and data augmentation techniques can also help mitigate class imbalance and improve generalization. Moreover, integrating multimodal data – such as user behavior, ratings, or temporal trends – could enhance aspect-based sentiment prediction beyond text analysis.

Further exploration into explainable AI (XAI) approaches is essential to improve model transparency and user trust, especially in sensitive fields like healthcare and customer analytics. Hybrid models that combine the interpretability of traditional algorithms with the contextual strength of transformers present another promising direction. Additionally, future research should investigate real-time ABSA frameworks capable of processing large-scale streaming data, enabling practical deployment in online review systems and customer support platforms.

To summarize, this study demonstrates that while traditional and deep learning approaches proved valuable insights into sentiment dynamics, transformer-based architectures like DistilBERT mark a significant advancement toward achieving balanced, context-aware sentiment understanding. Continued refinement through hybridization, interpretability, and cross-domain testing will pave the way for more accurately explainable, and robust sentiment analysis systems capable of supporting intelligent decision making across diverse real-world applications.

References

[1] Bo. Pang and Lillian. Lee, Opinion mining and sentiment analysis. Foundations and Trends® in information retrieval 2, no. 1–2, 2008.

[2] B. Liu, “Sentiment Analysis and Opinion Mining,” Morgan & Claypool Publishers, May 2022.

[3] M. Wankhade, A. C. S. Rao, and C. Kulkarni, “A survey on sentiment analysis methods, applications, and challenges,” Artificial Intelligence Review, vol. 55, no. 7, pp. 5731–5780, Oct. 2022, doi: 10.1007/s10462-022-10144-1

[4] N. A. Sharma, A. B. M. S. Ali, and M. A. Kabir, “A review of sentiment analysis: tasks, applications, and deep learning techniques,” International Journal of Data Science and Analytics. Springer Science and Business Media Deutschland GmbH, 2024. doi: 10.1007/s41060-024-00594-x.

[5] M. Ahmad, S. Aftab, I. Ali, and N. Hameed, “Hybrid Tools and Techniques for Sentiment Analysis: A Review,” International Journal of Multidisciplinary Sciences and Engineering, vol. 8, pp. 28–33, Jun. 2017

[6] L. Yue, W. Chen, X. Li, W. Zuo, and M. Yin, “A survey of sentiment analysis in social media,” Knowledge and Information Systems, vol. 60, no. 2, pp. 617–663, Aug. 2019, doi: 10.1007/s10115-018-1236-4.

[7] Nikhil Sanjay Suryawanshi, “Sentiment analysis with machine learning and deep learning: A survey of techniques and applications,” International Journal of Science and Research Archive, vol. 12, no. 2, pp. 005–015, Jul. 2024, doi: 10.30574/ijsra.2024.12.2.1205.

[8] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of Deep Bidirec-tional Transformers for Language Understanding,” Jun. 2019.

[9] R. Socher et al., “Recursive Deep Models for Semantic Compositionality Over a Sentiment Treebank,” Association for Computational Linguistics, Oct. 2013. [Online]. Available: http://nlp.stanford.edu/

[10] R. Feldman, “Techniques and applications for sentiment analysis: The main applications and challenges of one of the hottest research areas in computer science,” Communications of the ACM, vol. 56, no. 4. pp. 82–89, Apr. 2013. doi: 10.1145/2436256.2436274.

[11] A. Shathik and K. Prasad, “A Literature Review on Application of Sentiment Analysis Using Machine Learning Techniques,” International Journal of Applied Engineering and Management Letters (IJAEML) A Refereed International Journal of Srinivas University, vol. 4, no. 2, pp. 2581–7000, 2020, doi: 10.5281/zenodo.3977576.

[12] S. Chaturvedi, V. Mishra, and N. Mishra, Sentiment Analysis using Machine Learning for Business Intelligence. Institute of Electrical and Electronics Engineers, 2017.

[13] H. Huang, A. A. Zavareh, and M. B. Mustafa, “Sentiment Analysis in E-Commerce Platforms: A Review of Current Techniques and Future Directions,” IEEE Access, vol. 11. Institute of Electrical and Electronics Engineers Inc., pp. 90367–90382, 2023. doi: 10.1109/ACCESS.2023.3307308.

[14] M. A. Alonso, D. Vilares, C. Gómez-Rodríguez, J. Vilares, and G. Lys, “Sentiment Analysis for Fake News Detection,” 2021, doi: 10.3390/electronics.

[15] D. R. Kawade and Dr. K. S. Oza, “Sentiment Analysis: Machine Learning Approach,” International Journal of Engineering and Technology, vol. 9, no. 3, pp. 2183–2186, Jun. 2017, doi: 10.21817/ijet/2017/v9i3/1709030151.

[16] S. Yi and X. Liu, “Machine learning based customer sentiment analysis for recommending shoppers, shops based on customers’ review,” Complex and Intelligent Systems, vol. 6, no. 3, pp. 621–634, Oct. 2020, doi: 10.1007/s40747-020-00155-2.

[17] E. Deniz, H. Erbay, and M. Coşar, “Multi-Label Classification of E-Commerce Customer Reviews via Machine Learning,” Axioms, vol. 11, no. 9, Sep. 2022, doi: 10.3390/axioms11090436

[18] J. R. Saura, P. Palos-Sanchez, and A. Grilo, “Detecting indicators for startup business success: Sentiment analysis using text data mining,” Sustainability (Switzerland), vol. 11, no. 3, Feb. 2019, doi: 10.3390/su11030917.

[19] A. M. Alnasrawi, A. M. N. Alzubaidi, and A. A. Al-Moadhen, “Improving sentiment analysis using text network features within different machine learning algorithms,” Bulletin of Electrical Engineering and Informatics, vol. 13, no. 1, pp. 405–412, Feb. 2024, doi: 10.11591/eei.v13i1.5576.

[20] K. Mehta and S. P. Panda, “Sentiment Analysis on E-Commerce Apparels using Convolutional Neural Network,” International Journal of Computing, vol. 21, no. 2, pp. 234–241, 2022, doi: 10.47839/ijc.21.2.2592.

[21] X. Lin, “Sentiment Analysis of E-commerce Customer Reviews Based on Natural Language Processing,” in ACM International Conference Proceeding Series, Association for Computing Machinery, Apr. 2020, pp. 32–36. doi: 10.1145/3436286.3436293.

[22] H. Kundra, C. Vishnu, B. V. Vaishnavi, C. R. Kasireddy, G. Akash, and S. Jindam, “Digital Emotions using Sentiment Analysis for Predictive Insights on Customer Recommendations,” in INDISCON 2024 - 5th IEEE India Council International Subsections Conference: Science, Technology and Society, Institute of Electrical and Electronics Engineers Inc., 2024. doi: 10.1109/INDISCON62179.2024.10744376.

[23] H. Zhao, Z. Liu, X. Yao, and Q. Yang, “A machine learning-based sentiment analysis of online product reviews with a novel term weighting and feature selection approach,” Information Processing and Management, vol. 58, no. 5, Sep. 2021, doi: 10.1016/j.ipm.2021.102656.

[24] R. Vatambeti, S. V. Mantena, K. v.d. Kiran, M. Manohar, and C. Manjunath, “Twitter sentiment

analysis on online food services based on elephant herd optimization with hybrid deep learning technique," Cluster Computing, vol. 27, no. 1, pp. 655–671, Feb. 2024, doi: 10.1007/s10586-023-03970-7.

[25] R. K. Behera, M. Jena, S. K. Rath, and S. Misra, "Co-LSTM: Convolutional LSTM model for sentiment analysis in social big data," Information Processing and Management, vol. 58, no. 1, Jan. 2021, doi: 10.1016/j.ipm.2020.102435

[26] P. Ray and A. Chakrabarti, "A Mixed approach of Deep Learning method and Rule-Based method to improve Aspect Level Sentiment Analysis," Applied Computing and Informatics, vol. 18, no. 1–2, pp. 163–178, Jan. 2022, doi: 10.1016/j.aci.2019.02.002.

[27] P. K. Jain, V. Saravanan, and R. Pamula, "A Hybrid CNN-LSTM: A Deep Learning Approach for Consumer Sentiment Analysis Using Qualitative User-Generated Contents," ACM Trans-actions on Asian and Low-Resource Language Information Processing, vol. 20, no. 5, Sep. 2021, doi: 10.1145/3457206.

[28] I. Priyadarshini and C. Cotton, "A novel LSTM–CNN–grid search-based deep neural network for sentiment analysis," Journal of Supercomputing, vol. 77, no. 12, pp. 13911–13932, Dec. 2021, doi: 10.1007/s11227-021-03838-w.

[29] J. Jabbar, I. Urooj, W. JunSheng, and N. Azeem, Real-time Sentiment Analysis On E-Commerce Application. IEEE, 2019.

[30] Y. Li, J. Chan, G. Peko, and D. Sundaram, "An explanation framework and method for AI-based text emotion analysis and visualisation," Decision Support Systems, vol. 178, Mar. 2024, doi: 10.1016/j.dss.2023.114121.

[31] Tian, Y., Liu, C., Song, Y., Xia, F., & Zhang, Y. "Aspect-based sentiment analysis with context denoising." Findings of the Association for Computational Linguistics: NAACL 2024. June 2024.

[32] Yan, Z., Hsu, W., Lee, M. L., & Bartram-Shaw, D. "Modeling Complex Interactions in Long

Documents for Aspect-Based Sentiment Analysis." Proceedings of the 14th Workshop on Computational Approaches to Subjectivity, Sentiment, & Social Media Analysis. August 2024.

[33] Huang, D., Ren, L., & Li, Z. "Enhancing aspect-based sentiment analysis with linking words-guided emotional augmentation and hybrid learning." Neurocomputing 612 (2025): 128705.

[34] Aziz, K., Ji, D., Chakrabarti, P., Chakrabarti, T., Iqbal, M. S., & Abbasi, R. "Unifying aspect-based sentiment analysis BERT and multi-layered graph convolutional networks for comprehensive sentiment dissection." Scientific reports 14.1 (2024): 14646.

[35] Park, E. S., Dave, R., and Bhavsar, M. "Comparative Research in Sentiment Analysis Using Machine Learning Technique." Journal of Data Analysis and Information Processing 13.3 (2025): 269–280. https://doi.org/10.4236/jdaip.2025.133016

.

Appendix A

Code for Data Organization/Processing

The code in the appendix organizes the data, post which is ready for the feature extraction phase.

A.1 Data Extraction

```
# load dataset
!pip install ucimlrepo

from ucimlrepo import fetch_ucirepo

# fetch dataset
drug_reviews_druglib_com = fetch_ucirepo(id=461)

# data (as pandas dataframes)
X = drug_reviews_druglib_com.data.features

# metadata
print(drug_reviews_druglib_com.metadata)

# variable information
print(drug_reviews_druglib_com.variables)
```

A.2 Data Organization

From the dataset used for this research, each type of review corresponds to a specific aspect. The *benefitsReview* column represents the *Benefit* aspect, the *sideEffectsReview* column corresponds to the *Side Effects* aspect, and the *commentsReview* column captures the general or overall aspect. Furthermore, categorical columns serve as sentiment labels for each respective aspect: *effectiveness* is used as the sentiment label for the *Benefits* aspect with values as “Highly Effective”, or “Marginally Effective”, while *sideEffects* is the sentiment label for the *Side Effects* aspect with categories such as “Severe”, “Mild”, or “None”.

```
# Clean Reviews
import unicodedata, re
def basic_clean(s: str) -> str:
    if not isinstance(s, str):
        return ""
    s = unicodedata.normalize("NFKC", s)
    s = s.lower().strip()
    s = re.sub(r"\s+", " ", s)                    # collapse spaces
    s = re.sub(r"(.)\1{2,}", r"\1\1\1", s)          # cooool -> coool
    s = re.sub(r"http\S+|www\.\S+", "<url>", s)      # URLs
    s = re.sub(r"@\w+", "<user>", s)                # handles
    s = re.sub(r"\b\d+(\.\d+)?\b", "<num>", s)       # numbers -> <num>
    # keep ! ? ' for negation/sentiment
    return s

from collections import Counter
import numpy as np
```

```
def class_stats(y, labels=None, name=""):
    cnt = Counter(y)
    n = sum(cnt.values())
    labels = labels or sorted(cnt.keys())
    print(f"[{name}] N={n} | " + " | ".join(f"{c}:{cnt[c]} ({cnt[c]/n:.1%})" for c in labels))
    # weights: inverse-frequency normalized to mean=1
    freqs = np.array([cnt[c] for c in labels], dtype=float)
    w = (freqs.mean() / freqs)
    return {c: float(w[i]) for i, c in enumerate(labels)}

# Map numeric ratings → sentiment (neg, neu, pos)
def map_rating(r):
    if r <= 3:    return "negative"
    elif r <= 7:  return "neutral"
    elif r <= 10: return "positive"
    else: return None
X['sentiment'] = X['rating'].apply(map_rating)

# Map Side Effect
def map_side_effects(val):
    if "No Side Effects" in val:                return "positive"
    elif "Mild Side Effects" in val:            return "neutral"
    elif "Moderate Side Effects" in val:        return "negative"
    elif "Severe Side Effects" in val:          return "negative"
    elif "Extremely Severe Side Effects" in val:  return "negative"
    else: return None
X['side_sentiment'] = X['sideEffects'].apply(map_side_effects)
```

```
# Map Effectiveness
def map_effectiveness(val):
    if "Highly Effective" in val:           return "positive"
    elif "Considerably Effective" in val:   return "positive"
    elif "Moderately Effective" in val:     return "neutral"
    elif "Marginally Effective" in val:     return "neutral"
    elif "Ineffective" in val:              return "negative"
    else: return None
X['benefit_sentiment'] = X['effectiveness'].apply(map_effectiveness)

df = X.copy()
df['comments_clean'] = df['commentsReview'].apply(basic_clean)
df['benefits_clean'] = df['benefitsReview'].apply(basic_clean)
df['side_clean']     = df['sideEffectsReview'].apply(basic_clean)
df['label_comments'] = df['rating'].apply(map_rating)
df['label_benefits'] = df['effectiveness'].apply(map_effectiveness)
df['label_side']     = df['sideEffects'].apply(map_side_effects)

df = df.dropna(subset=['comments_clean','benefits_clean','side_clean',
                       'label_comments','label_benefits','label_side']).reset_index(drop=True)
```

A.3 Feature Extraction

Text data from each aspect was converted into numerical form for model training. Traditional models used TF-IDF at both word and character levels to capture important terms, while deep learning models applied word embeddings to represent contextual meaning. The DistilBERT model used pre-trained contextual embeddings, allowing it to better understand sentiment within each aspect.

TF-IDF

```
feats = ColumnTransformer([
    ("tfidf_word", TfidfVectorizer(ngram_range=(1,2), min_df=2, max_features=100_000), "commentsReview"),
    ("tfidf_char", TfidfVectorizer(analyzer="char", ngram_range=(3,5), min_df=2, max_features=100_000), "commentsReview"),
], remainder="drop", verbose_feature_names_out=False)

clf = Pipeline([
    ("feats", feats),
    ("svm", LinearSVC(class_weight="balanced", random_state=42))
])
clf.fit(X_train, y_train)
```

Keras tokenization + Embedding

```
# label encode
le = LabelEncoder()
y = to_categorical(le.fit_transform(labels), num_classes=len(le.classes_))

# tokenize
MAX_WORDS = 30000
MAX_LEN = 200
tok = Tokenizer(num_words=MAX_WORDS, oov_token="<unk>")
tok.fit_on_texts(texts)
X = pad_sequences(tok.texts_to_sequences(texts), maxlen=MAX_LEN)
```

```
# simple CNN-BiLSTM model (feature extraction happens via Embedding/Conv/LSTM)
inp = Input(shape=(MAX_LEN,))
x = Embedding(input_dim=min(MAX_WORDS, len(tok.word_index)+1), output_dim=128)(inp)
x = Conv1D(128, 5, padding="same", activation="relu")(x)
x = Bidirectional(LSTM(64, return_sequences=True))(x)
x = GlobalMaxPooling1D()(x)
out = Dense(y.shape[1], activation="softmax")(x)
model = Model(inp, out)
model.compile(optimizer="adam", loss="categorical_crossentropy", metrics=["accuracy"])
model.summary()
model.fit(X, y, epochs=3, batch_size=64, validation_split=0.2, shuffle=True)
```

DistilBERT tokenization

```
tokenizer = AutoTokenizer.from_pretrained("distilbert-base-uncased")


def encode_pair(review_texts, aspect_name):
    # aspect_name e.g., "Benefits", "Side Effects", or "Comments"
    return tokenizer(
        review_texts,
        [aspect_name] * len(review_texts),
        padding=True,
        truncation=True,
        max_length=256,
        return_tensors="pt"
    )
```

Appendix B

Code for Model Training and Testing

For training and testing the models, the dataset was randomly divided into two parts – 80% for training and 20% for testing. The training data was used to let the models learn the patterns and relationship between the text and sentiment labels, while the testing data was used to evaluate their performance. All models were developed in Python, using libraries such as scikit-learn, Tensorflow, and Transformers to perform classification and assess accuracy, precision, recall, and F1-score.

B.1 Traditional Machine Learning Model

```
import numpy as np
import pandas as pd
from dataclasses import dataclass
from typing import Dict, List, Any
from sklearn.preprocessing import LabelEncoder
from sklearn.pipeline import Pipeline
from sklearn.model_selection import StratifiedKFold, GridSearchCV
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics import f1_score, accuracy_score
from sklearn.svm import LinearSVC, SVC
from sklearn.multiclass import OneVsRestClassifier
from xgboost import XGBClassifier
import warnings
warnings.filterwarnings("ignore", category=UserWarning)


# Expect columns in your df
ASPECTS: Dict[str, Dict[str, str]] = {
    "Comments":  {"text": "comments_clean",  "label": "label_comments"},
    "Benefits":  {"text": "benefits_clean",  "label": "label_benefits"},
```

```
    "SideEffects":   {"text": "side_clean",  "label": "label_side"},
}

DEFAULT_MODELS = ["SVM","SVC","XGBoost"]

def _safe_array(x):
    if isinstance(x, (pd.Series, pd.Index)):
        return x.to_numpy()
    if isinstance(x, list):
        return np.asarray(x)
    return x

def _fit_label_encoder(y_str: np.ndarray):
    le = LabelEncoder()
    y_enc = le.fit_transform(y_str)
    return le, y_enc

def _effective_n_splits(y: np.ndarray, desired_splits: int) -> int:
    _, counts = np.unique(y, return_counts=True)
    max_allowed = counts.min()
    return max(2, min(desired_splits, int(max_allowed)))

def _build_pipeline(model_name: str, n_classes: int):
    """
    Build (pipeline, param_grid) for the given model.
    All pipelines share TF-IDF -> clf with step name 'clf'.
    """
    tfidf = TfidfVectorizer(max_features=30000, ngram_range=(1, 3),
                            analyzer="word", sublinear_tf=True)

    if model_name == "SVM":
        clf = OneVsRestClassifier(LinearSVC(
            class_weight="balanced",
```

```
        random_state=42,
    ))
    pipe = Pipeline([("tfidf", tfidf), ("clf", clf)])
    param_grid = {
        "clf__estimator__C": [0.1, 1.0, 10.0],
    }

elif model_name == "SVC":
    clf = OneVsRestClassifier(SVC(
        kernel="rbf",
        class_weight="balanced",
        probability=True,
        random_state=42,
    ))
    pipe = Pipeline([("tfidf", tfidf), ("clf", clf)])
    param_grid = {
        "clf__estimator__C": [1.0, 4.0],
        "clf__estimator__gamma": ["scale", 0.1],
    }

elif model_name == "XGBoost":
    clf = XGBClassifier(
        objective="multi:softprob",
        num_class=n_classes,
        eval_metric="mlogloss",
        tree_method="hist",
        max_depth=6, learning_rate=0.1, n_estimators=400,
        subsample=0.8, colsample_bytree=0.8, reg_lambda=1.0,
        n_jobs=-1,
        random_state=42,
    )
    pipe = Pipeline([("tfidf", tfidf), ("clf", clf)])
    param_grid = {
```

```
            "clf__max_depth": [4, 6],
            "clf__learning_rate": [0.05, 0.1],
            "clf__subsample": [0.8, 1.0],
            "clf__colsample_bytree": [0.8, 1.0],
            "clf__min_child_weight": [1, 3],
        }

    else:
        raise ValueError(f"Unknown model: {model_name}")

    return pipe, param_grid

@dataclass
class FoldResult:
    fold_idx: int
    best_params: Dict[str, Any]
    macro_f1: float
    accuracy: float

@dataclass
class AspectResult:
    aspect: str
    model: str
    n_samples: int
    labels: List[str]
    folds: List[FoldResult]

    @property
    def macro_f1_mean(self) -> float:
        return float(np.mean([f.macro_f1 for f in self.folds])) if self.folds else np.nan

    @property
    def macro_f1_std(self) -> float:
```

```
        return float(np.std([f.macro_f1 for f in self.folds])) if self.folds else np.nan

    @property
    def acc_mean(self) -> float:
        return float(np.mean([f.accuracy for f in self.folds])) if self.folds else np.nan

    @property
    def acc_std(self) -> float:
        return float(np.std([f.accuracy for f in self.folds])) if self.folds else np.nan


def nested_cv_for_aspect(
    texts: np.ndarray,
    y_str: np.ndarray,
    model_name: str,
    outer_splits: int = 5,
    inner_splits: int = 3,
    random_state: int = 42,
) -> Tuple[AspectResult, LabelEncoder]:
    """
    Nested cross-validation for one aspect.
    Stores y_true, y_pred, and confusion matrix per fold.
    """
    texts = _safe_array(texts).astype(str)
    y_str = _safe_array(y_str).astype(str)

    le = LabelEncoder()
    y = le.fit_transform(y_str)
    n_classes = len(le.classes_)

    outer_k = _effective_n_splits(y, outer_splits)
    inner_k = _effective_n_splits(y, inner_splits)

    pipe, param_grid = _build_pipeline(model_name, n_classes)
```

```
outer_cv = StratifiedKFold(n_splits=outer_k, shuffle=True, random_state=random_state)
inner_cv = StratifiedKFold(n_splits=inner_k, shuffle=True, random_state=random_state)

gs = GridSearchCV(
    estimator=pipe,
    param_grid=param_grid,
    cv=inner_cv,
    scoring="f1_macro",
    n_jobs=-1,
    verbose=0,
    error_score="raise",
)

fold_results: List[FoldResult] = []

for fidx, (tr, te) in enumerate(outer_cv.split(texts, y), start=1):
    gs.fit(texts[tr], y[tr])
    y_pred = gs.predict(texts[te])

    macro_f1 = f1_score(y[te], y_pred, average="macro")
    acc = accuracy_score(y[te], y_pred)
    cm = confusion_matrix(y[te], y_pred, labels=list(range(n_classes)))  # ✅ now defined

    fold_results.append(FoldResult(
        fold_idx=fidx,
        best_params=gs.best_params_,
        macro_f1=macro_f1,
        accuracy=acc,
        y_true=y[te],
        y_pred=y_pred,
        confmat=cm
    ))
```

```
        print(f"[{model_name}] Fold {fidx}/{outer_k} — macro-F1: {macro_f1:.4f} | acc: {acc:.4f} | best:
{gs.best_params_}")


    ar = AspectResult(
        aspect="",  # filled in later by run_all_aspects
        model=model_name,
        n_samples=len(texts),
        labels=list(le.classes_),
        folds=fold_results,
    )
    return ar, le


def run_all_aspects(
    df: pd.DataFrame,
    aspects: Dict[str, Dict[str, str]] = ASPECTS,
    models_to_run: List[str] = DEFAULT_MODELS,
    min_rows: int = 50
):
    """
    Runs nested CV for each requested model on each aspect (dict config).
    Returns nested dict: results[aspect][model] -> AspectResult
    """
    results: Dict[str, Dict[str, AspectResult]] = {}

    for aspect, cfg in aspects.items():
        text_col = cfg["text"]
        label_col = cfg["label"]

        sub = df[[text_col, label_col]].dropna().reset_index(drop=True)
        texts = sub[text_col].astype(str).to_numpy()
        y_str = sub[label_col].astype(str).to_numpy()

        print("\n" + "=" * 28, f"{aspect.upper()}", "=" * 28)
```

```
        print(f"N = {len(sub)} | labels: {sorted(list(pd.Series(y_str).unique()))}")

        if len(sub) < min_rows:
            print(f"Skipping (too few rows < {min_rows}).")
            continue

        results[aspect] = {}

        for model_name in models_to_run:
            print(f"\n===== {model_name} =====")
            try:
                ar, _ = nested_cv_for_aspect(texts, y_str, model_name=model_name)
                ar.aspect = aspect
                results[aspect][model_name] = ar

                print("-" * 72)
                print(f"{aspect} — {model_name}:")
                print(f"macro-F1: {ar.macro_f1_mean:.4f} | "
                      f"acc: {ar.acc_mean:.4f}")
                print(f"labels: {ar.labels}")
            except Exception as e:
                print("-" * 72)
                print(f"{aspect} — {model_name} FAILED with error:\n{e}")

    return results
```

B.2 Deep Learning Model: CNN-BiLSTM

```
import tensorflow as tf
import numpy as np
from sklearn.model_selection import KFold
from sklearn.preprocessing import LabelEncoder
from tensorflow.keras.utils import to_categorical
```

```
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras import layers, regularizers, Model
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau, ModelCheckpoint
from sklearn.model_selection import train_test_split

SEED = 42
def set_seed(seed=SEED):
    random.seed(SEED)
    np.random.seed(SEED)
    tf.random.set_seed(SEED)
set_seed(SEED)

# Text to IDs
def prepare_texts(texts, max_words=30000, max_len=200):
    tok = Tokenizer(num_words=max_words, oov_token="<OOV>")
    tok.fit_on_texts(texts)
    seqs = tok.texts_to_sequences(texts)
    padded = pad_sequences(seqs, maxlen=max_len, padding="post", truncating="post")
    vocab_size = min(max_words, len(tok.word_index) + 1)
    return tok, padded, vocab_size

def counts(y_int, le):
    c = Counter(y_int)
    out = {}
    for cls in ["negative", "neutral", "positive"]:
        if cls in le.classes_:
            idx = np.where(le.classes_ == cls)[0]
            if len(idx): out[cls] = int(c.get(idx[0], 0))
    # Add any remaining classes (if any)
```

```
    for i, cls in enumerate(le.classes_):
        if cls not in out:
            out[cls] = int(c.get(i, 0))
    return out


# GloVe Loader
def load_glove_embedding_matrix(tokenizer, vocab_size, embedding_dim, glove_txt_path, trainable=False):
    """
    glove_txt_path: e.g., '/content/glove.6B.100d.txt'
    embedding_dim must match the file (50/100/200/300)
    """
    embeddings_index = {}
    with open(glove_txt_path, 'r', encoding='utf-8', errors='ignore') as f:
        for line in f:
            values = line.rstrip().split(" ")
            word = values[0]
            coefs = np.asarray(values[1:], dtype='float32')
            embeddings_index[word] = coefs

    embedding_matrix = np.random.normal(scale=0.6, size=(vocab_size, embedding_dim)).astype(np.float32)
    for word, i in tokenizer.word_index.items():
        if i >= vocab_size:
            continue
        vec = embeddings_index.get(word)
        if vec is not None:
            embedding_matrix[i] = vec
    return embedding_matrix, trainable

def build_cnn_bilstm(vocab_size, max_len, n_classes, emb_dim=100, trainable=True, dropout=0.2):
    inp = layers.Input(shape=(max_len,), name="tokens")
```

```
    emb = layers.Embedding(vocab_size, emb_dim, name="emb", mask_zero=False, trainable=trainable)(inp)  # changed
    x = layers.Conv1D(128, 3, padding="same", activation="relu")(emb)
    x = layers.MaxPooling1D(2)(x)
    x = layers.Bidirectional(layers.LSTM(128, return_sequences=True))(x)
    x = layers.GlobalMaxPool1D()(x)
    x = layers.Dropout(dropout)(x)
    out = layers.Dense(n_classes, activation="softmax")(x)
    model = models.Model(inp, out)
    model.compile(optimizer=tf.keras.optimizers.Adam(1e-3),
                  loss="sparse_categorical_crossentropy", metrics=["accuracy"])
    return model


# Gradient-based token importance
@tf.function
def _grad_wrt_emb(model, token_batch):
    """
    Returns gradients of the predicted logit wrt the embedding outputs for each sample.
    """
    with tf.GradientTape() as tape:
        # get embedding output by re-calling the embedding layer explicitly
        emb_layer = model.get_layer("emb")
        emb_out = emb_layer(token_batch)  # [B, L, D]
        tape.watch(emb_out)

        # forward pass from conv onward by swapping the input
        x = emb_out
        for lyr in model.layers:
            if isinstance(lyr, tf.keras.layers.InputLayer) or lyr.name == "emb":
                continue
            x = lyr(x)
```

```
        probs = x  # softmax
        pred_ids = tf.argmax(probs, axis=1)  # [B]

        # gather predicted logit per sample
        bsz = tf.shape(probs)[0]
        idx0 = tf.range(bsz, dtype=pred_ids.dtype) # ensure same dtype
        gather_idx = tf.stack([idx0, pred_ids], axis=1)
        pred_logits = tf.gather_nd(probs, gather_idx)

    grads = tape.gradient(pred_logits, emb_out)     # [B, L, D]
    return grads, pred_ids

def token_saliency(model, tokenizer, text, max_len=200, agg="l2"):
    """
    Returns token list and importance scores for a single text.
    agg: 'l2' for ||grad ⊙ emb|| per token, or 'sum' for sum of absolute values.
    """
    seq = vectorize(tokenizer, [text], max_len=max_len)
    grads, pred_ids = _grad_wrt_emb(model, tf.constant(seq))
    emb = model.get_layer("emb")(tf.constant(seq))
    contrib = grads * emb  # grad ⊙ emb

    if agg == "l2":
        token_scores = tf.norm(contrib[0], ord=2, axis=-1).numpy()
    else:
        token_scores = tf.reduce_sum(tf.abs(contrib[0]), axis=-1).numpy()

    # map ids -> tokens (ignore padding=0)
    inv_vocab = {v:k for k,v in tokenizer.word_index.items() if v < tokenizer.num_words}
```

```
    ids = seq[0]
    tokens = [inv_vocab.get(i, "<pad>") if i>0 else "<pad>" for i in ids]
    return tokens, token_scores, int(pred_ids.numpy()[0])

def print_top_tokens(model, tokenizer, text, label_names, k=10, max_len=200):
    toks, scores, pred = token_saliency(model, tokenizer, text, max_len=max_len)
    pairs = [(t, s) for t,s in zip(toks, scores) if t not in ("<pad>",)]
    pairs.sort(key=lambda x: x[1], reverse=True)
    print(f"Pred: {label_names[pred]} | Top-{k} tokens: {[t for t,_ in pairs[:k]]}")

def aggregate_top_tokens(model, tokenizer, texts, y_pred_ids, label_names, per_class_k=15, max_len=200):
    """
    Aggregate top tokens across a sample of texts, per predicted class.
    """
    from collections import defaultdict, Counter
    buckets = defaultdict(list)
    for i, txt in enumerate(texts):
        toks, scores, pred = token_saliency(model, tokenizer, txt, max_len=max_len)
        pairs = [(t, s) for t,s in zip(toks, scores) if t not in ("<pad>",)]
        pairs.sort(key=lambda x: x[1], reverse=True)
        buckets[pred].extend([t for t,_ in pairs[:per_class_k]])
    # count & show top
    out = {}
    for cls_id, toks in buckets.items():
        cnt = Counter(toks).most_common(per_class_k)
        out[label_names[cls_id]] = cnt
    return out

# Per-aspect Runner
def run_cnn_bilstm_sharp(df, text_col, label_col,
```

```
                    max_len=200, num_words=50000, emb_dim=100,
                    epochs=5, batch_size=64, verbose=1, random_state=42):
    sub = df.dropna(subset=[text_col, label_col]).reset_index(drop=True)
    texts = sub[text_col].astype(str).tolist()
    labels = sub[label_col].astype(str).values

    le = LabelEncoder(); y = le.fit_transform(labels)
    Xtr_txt, Xte_txt, ytr, yte = train_test_split(texts, y, test_size=0.2,
                                        random_state=random_state, stratify=y)
    tok = build_tokenizer(Xtr_txt, num_words=num_words)
    Xtr = vectorize(tok, Xtr_txt, max_len=max_len)
    Xte = vectorize(tok, Xte_txt, max_len=max_len)

    model = build_cnn_bilstm(vocab_size=min(num_words, len(tok.word_index)+1),
                         max_len=max_len, n_classes=len(le.classes_), emb_dim=emb_dim)
    cb = [tf.keras.callbacks.EarlyStopping(patience=2, restore_best_weights=True, monitor="val_accuracy")]
    model.fit(Xtr, ytr, validation_split=0.1, epochs=epochs, batch_size=batch_size, callbacks=cb, verbose=verbose)

    ypred = model.predict(Xte, verbose=0).argmax(1)
    print("\n=== CNN-BiLSTM — Test Report ===")
    print(classification_report(yte, ypred, target_names=list(le.classes_), digits=3, zero_division=0))

    # SHARP: per-example & aggregate token importances
    print("\n[Sharp] Example token importances:")
    for i in range(min(3, len(Xte_txt))):
        print_top_tokens(model, tok, Xte_txt[i], le.classes_, k=10, max_len=max_len)

    print("\n[Sharp] Aggregate top tokens per predicted class:")
    agg = aggregate_top_tokens(model, tok, Xte_txt[:200], ypred[:200], le.classes_, per_class_k=20,
max_len=max_len)
    for cls, pairs in agg.items():
```

```
        print(f"  - {cls}: {[t for t,_ in pairs[:15]]}")


    return {"model": model, "tokenizer": tok, "label_encoder": le, "report": classification_report(yte, ypred,
target_names=list(le.classes_), output_dict=True, zero_division=0), "agg_top_tokens": agg}


def run_cnn_bilstm_all_aspects(df):
    results = {}
    for a, cfg in ASPECTS.items():
        print("\n" + "="*30)
        print(f"=== {a} — CNN-BiLSTM Sharp Analysis ===")
        print("="*30)
        results[a] = run_cnn_bilstm_sharp(df, cfg["text_col"], cfg["label_col"])
    return results



def build_cnn_bilstm_model(input_length, vocab_size, num_classes=3, embedding_dim=100):
    model = tf.keras.Sequential([
        tf.keras.layers.Embedding(vocab_size, embedding_dim, input_length=input_length),
        tf.keras.layers.Conv1D(128, 5, activation="relu"),
        tf.keras.layers.MaxPooling1D(2),
        tf.keras.layers.Bidirectional(tf.keras.layers.LSTM(64)),
        tf.keras.layers.Dense(64, activation="relu"),
        tf.keras.layers.Dense(num_classes, activation="softmax")
    ])
    model.compile(loss="categorical_crossentropy", optimizer="adam", metrics=["accuracy"])
    return model


def run_kfold(df, aspect_name, col_text, col_label,
                  k=5, epochs=3, batch_size=32,
                  max_words=30000, max_len=200):
    print(f"=== {aspect_name.upper()} Aspect - CNN-BiLSTM ===")
```

```
texts_all = df[col_text].astype(str).values
labels_all = df[col_label].astype(str).values

kf = KFold(n_splits=k, shuffle=True, random_state=42)
accs, reports, confmats = [], [], []

for fold, (tr, te) in enumerate(kf.split(texts_all), start=1):
    tr_texts, te_texts = texts_all[tr], texts_all[te]
    tr_labels, te_labels = labels_all[tr], labels_all[te]

    # Tokenizer on train
    tok, Xtr, vocab_size = prepare_texts(tr_texts, max_words=max_words, max_len=max_len)
    # Use same tokenizer for test
    seqs_te = tok.texts_to_sequences(te_texts)
    Xte = pad_sequences(seqs_te, maxlen=max_len, padding="post", truncating="post")

    # Encode labels
    le = LabelEncoder()
    ytr_int = le.fit_transform(tr_labels)
    yte_int = le.transform(te_labels)
    ytr = to_categorical(ytr_int, num_classes=len(le.classes_))
    yte = to_categorical(yte_int, num_classes=len(le.classes_))

    model = build_cnn_bilstm_model(input_length=Xtr.shape[1],
                                   vocab_size=vocab_size,
                                   num_classes=ytr.shape[1])

    print(f"\n----- Fold {fold} -----")
    model.fit(Xtr, ytr, validation_data=(Xte, yte),
```

```
            epochs=epochs, batch_size=batch_size, verbose=0)

        y_pred = model.predict(Xte, batch_size=batch_size, verbose=0)
        y_pred_cls = np.argmax(y_pred, axis=1)
        y_true = np.argmax(yte, axis=1)

        acc = accuracy_score(y_true, y_pred_cls); accs.append(acc)
        print(f"Accuracy: {acc:.4f}")

        rep = classification_report(y_true, y_pred_cls,
                                    target_names=list(le.classes_),
                                    digits=4, zero_division=0)
        print("Classification Report:\n", rep); reports.append(rep)

        cm = confusion_matrix(y_true, y_pred_cls)
        print("Confusion Matrix:\n", cm); confmats.append(cm)

    print(f"\nMean Accuracy over {k} folds: {np.mean(accs):.4f}")
    return accs, reports, confmats

def run_aspects_kfold(data_dict, vocab_size, k=5, epochs=3, batch_size=32):
    for aspect, pack in data_dict.items():
        print(f"=== {aspect.upper()} Aspect - CNN-BiLSTM ===")

        X = pack["X"]; y = pack["y"]
        assert X.ndim == 2, f"{aspect}: X must be [N, seq_len]"
        assert y.ndim == 2 and y.shape[1] == 3, f"{aspect}: y must be [N, 3] one-hot"

        accs, reports, confmats = run_kfold(
            X, y, vocab_size=vocab_size, k=k, epochs=epochs, batch_size=batch_size
```

```
        )
        print(f">>> {aspect.upper()} Aspect — Mean Accuracy: {np.mean(accs):.4f}")

# === Comments Aspect — CNN-BiLSTM ===
accs_c, reports_c, confmats_c = run_kfold(
    df,
    aspect_name="Comments",
    col_text="comments_clean",
    col_label="label_comments",
    k=5, epochs=3, batch_size=32,
    max_words=30000, max_len=200
)

# === Benefits Aspect — CNN-BiLSTM ===
accs_b, reports_b, confmats_b = run_kfold(
    df,
    aspect_name="Benefits",
    col_text="benefits_clean",
    col_label="label_benefits",
    k=5, epochs=3, batch_size=32,
    max_words=30000, max_len=200
)

# === Side Effects Aspect — CNN-BiLSTM ===
accs_s, reports_s, confmats_s = run_kfold(
    df,
    aspect_name="SideEffects",
    col_text="side_clean",
    col_label="label_side",
    k=5, epochs=3, batch_size=32,
```

```
    max_words=30000, max_len=200
)
```

B.3 Transformer-Based Model: DistilBERT

```
!pip -q install "transformers>=4.42" "datasets>=2.20" "accelerate>=0.33" evaluate scikit-learn
import numpy as np
import pandas as pd
import random, torch, evaluate
from sklearn.model_selection import StratifiedShuffleSplit
from sklearn.metrics import classification_report, confusion_matrix
from transformers import (
    DistilBertTokenizerFast,
    DistilBertForSequenceClassification,
    TrainingArguments,
    Trainer,
    DataCollatorWithPadding,
)
from datasets import Dataset, DatasetDict

LABELS = ["negative","neutral","positive"]
label2id = {k:i for i,k in enumerate(LABELS)}
id2label = {i:k for k,i in label2id.items()}

def make_pairs(frame: pd.DataFrame) -> pd.DataFrame:
    rows = []
    # comments
    rows.append(pd.DataFrame({
        "text": frame["comments_clean"].values,
        "aspect": ["comments"]*len(frame),
        "label_str": frame["label_comments"].values
    }))
    # benefits
    rows.append(pd.DataFrame({
```

```
        "text": frame["benefits_clean"].values,
        "aspect": ["benefits"]*len(frame),
        "label_str": frame["label_benefits"].values
    }))
    # side effects
    rows.append(pd.DataFrame({
        "text": frame["side_clean"].values,
        "aspect": ["side effects"]*len(frame),
        "label_str": frame["label_side"].values
    }))
    out = pd.concat(rows, ignore_index=True)
    out = out[out["label_str"].isin(LABELS)].reset_index(drop=True)
    out["label"] = out["label_str"].map(label2id)
    return out
pairs = make_pairs(df)
print("Pairs shape:", pairs.shape)
print(pairs.head())

# Train/Validation split (stratified)
# Stratify on both aspect and label so each section keeps its class miz
pairs["strat_key"] = pairs["aspect"].astype(str) + "___" + pairs["label_str"].astype(str)
splitter = StratifiedShuffleSplit(n_splits=1, test_size=0.2, random_state=42)
train_idx, test_idx = next(splitter.split(pairs, pairs["strat_key"]))
train_df = pairs.iloc[train_idx].drop(columns=["strat_key"]).reset_index(drop=True)
test_df  = pairs.iloc[test_idx].drop(columns=["strat_key"]).reset_index(drop=True)
print("Train size:", len(train_df), " Test size:", len(test_df))
print(train_df["aspect"].value_counts())
print(test_df["aspect"].value_counts())

# Tokenizer
tokenizer = DistilBertTokenizerFast.from_pretrained("distilbert-base-uncased")

def tok_fn(batch):
```

```
    # sentence-pair: [CLS] review [SEP] aspect [SEP]
    return tokenizer(
        batch["text"],
        text_pair=batch["aspect"],
        truncation=True,
        max_length=256,
    )

hf_train = Dataset.from_pandas(train_df[["text","aspect","label"]])
hf_test  = Dataset.from_pandas(test_df[["text","aspect","label"]])
hf_ds = DatasetDict({"train": hf_train, "test": hf_test}).map(tok_fn, batched=True)

data_collator = DataCollatorWithPadding(tokenizer=tokenizer)

# Build Model
model = DistilBertForSequenceClassification.from_pretrained(
    "distilbert-base-uncased",
    num_labels=len(LABELS),
    id2label=id2label,
    label2id=label2id,
)

# Train
training_args = TrainingArguments(
    output_dir="./distilbert_absa_pairs",
    learning_rate=2e-5,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=32,
    num_train_epochs=5,
    weight_decay=0.01,
    eval_strategy="epoch",
    save_strategy="epoch",
    load_best_model_at_end=True,
```

```
    metric_for_best_model="f1_macro",
    seed=42,
    logging_steps=50,
    report_to="none",
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=hf_ds["train"],
    eval_dataset=hf_ds["test"],
    tokenizer=tokenizer,
    data_collator=data_collator,
    compute_metrics=compute_metrics,
)

trainer.train()

# Metrics
acc_metric = evaluate.load("accuracy")
prec_metric = evaluate.load("precision")
rec_metric = evaluate.load("recall")
f1_metric = evaluate.load("f1")

def compute_metrics(eval_pred):
    logits, labels = eval_pred
    preds = np.argmax(logits, axis=1)
    out = {}
    out.update(acc_metric.compute(predictions=preds, references=labels))
    # macro (average='macro'), micro ('micro'), and weighted
    out["precision_macro"] = prec_metric.compute(predictions=preds, references=labels,
average="macro")["precision"]
    out["recall_macro"]    = rec_metric.compute(predictions=preds, references=labels, average="macro")["recall"]
    out["f1_macro"]        = f1_metric.compute(predictions=preds, references=labels, average="macro")["f1"]
```

```
    out["f1_weighted"]     = f1_metric.compute(predictions=preds, references=labels, average="weighted")["f1"]
    return out

# Overall Test Metrics
test_out = trainer.predict(hf_ds["test"])
test_preds = np.argmax(test_out.predictions, axis=1)
print("\n=== Overall Test Classification Report ===")
print(classification_report(test_df["label"], test_preds, target_names=LABELS, digits=4))

# Section Metrics
def section_report(section_name: str):
    mask = (test_df["aspect"] == section_name)
    y_true = test_df.loc[mask, "label"].values
    # Re-run prediction only on that subset for neatness (or slice from test_preds)
    sub_ds = hf_ds["test"].select(np.where(mask)[0].tolist())
    sub_out = trainer.predict(sub_ds)
    y_pred = np.argmax(sub_out.predictions, axis=1)

    print(f"\n=== {section_name.capitalize()} — Test Report ===")
    print(classification_report(y_true, y_pred, target_names=LABELS, digits=4))

    cm = confusion_matrix(y_true, y_pred, labels=[0,1,2])
    cm_df = pd.DataFrame(cm, index=[f"true_{l}" for l in LABELS], columns=[f"pred_{l}" for l in LABELS])
    print("\nConfusion Matrix:")
    print(cm_df)

for sec in ["comments", "benefits", "side effects"]:
    section_report(sec)
```